

Form4

Capstone Design Implementation

Title :

Mobile Sales Canvassing Application

GROUP MEMBER:

No.	Student Name	Student ID
1.	Dannis Surya Ridfan	001202300188
2.	Pieter Utomo Wijaya	001202300096
3.	Muhammad Ibnul Aroby	001202300086

Advisor : Abdul Ghofir

Submitted for

Capstone Design Project

to Faculty of Computer Science

President University

TABLE OF CONTENT

STATEMENT OF ORIGINALITY

SCREENSHOT OF ZEROGPT

PART 4 — IMPLEMENTATION

DESIGNS IMPLEMENTATION

1. Authentication (Login & JWT)

2. Dashboard & Visits

3. Route Planning & GPS

4. Order Taking

5. Stock Management

6. Collections

7. Analytics & Merchandising

8. Admin & Outlet Management

Database / UI / Hardware / Integration

PRODUCT DISPLAY

COMPONENT COST ANALYSIS

FUNCTIONAL TESTING

MANUAL GUIDE

REFERENCES

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone
Design Project stated below:

Name :

1. Dannis Surya Ridfan — 001202300188
2. Pieter Utomo Wijaya — 001202300096
3. Muhammad Ibnul Aroby — 001202300086

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled "Mobile Sales Canvassing Application"
is to the best of my knowledge and belief, an original piece of work based on sound academic
principles. If there is any plagiarism detected in this final project, I am willing to be personally
responsible for the consequences of these acts of plagiarism and will accept the sanctions against
these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another
university to obtain a degree.

Signer 1	Signer 2	Signer 3
Dannis Surya Ridfan 001202300188	Pieter Utomo Wijaya 001202300096	Muhammad Ibnul Aroby 001202300086

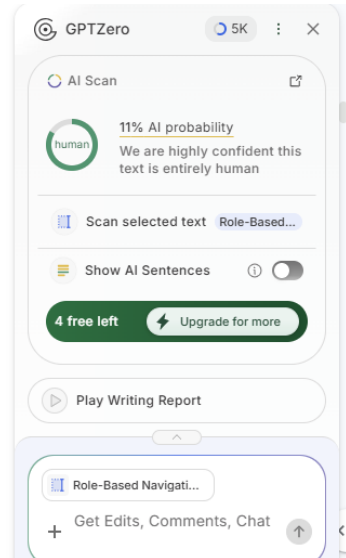
Cikarang, 30 May 2026

SCREENSHOT OF ZEROGPT

After authentication, RoleBasedNavigator reads the stored role from SharedPreferences via AuthService.getUserRole(). Administrators and managers are routed to AdminDashboard for master data and sales team management. Supervisors see SupervisorDashboard with team-level KPIs from GET /api/supervisor/team-dashboard. Sales representatives enter MainNavigation with a drawer containing Dashboard, Visits, Route Planning, GPS Tracking, Order Taking, Stock, Collections, Analytics, Merchandising, Outlet Management, and Settings.

The authorize (roles) middleware on protected routes enforces server-side RBAC. For example, GET /api/admin/sales-reps requires admin or manager role; a rep receives HTTP 403 Forbidden if attempting admin endpoints. Combined with JWT employee_id filtering, this implements defense in depth for per-user data isolation required by the project.

Listing 1.2.1 — Flutter: RoleBasedNavigator (main.dart)



PART 4

IMPLEMENTATION

Consists of:

- DESIGNS IMPLEMENTATION
- PRODUCT DISPLAY
- COMPONENT COST ANALYSIS
- FUNCTIONAL TESTING
- MANUAL GUIDE

DESIGNS IMPLEMENTATION

Functions / Procedure / Class Implementation

Before starting development, the team prepared the following environment based on Form 3 and the actual technology stack used in the repository `sales_canvassing_app` and `sales_canvassing_backend`:

- Flutter SDK 3.x and Dart 3.x — cross-platform mobile UI
- Android Studio or Visual Studio Code — IDE, emulator, and debugging
- Node.js 18+ and npm — backend runtime and package management
- PostgreSQL 14+ and pgAdmin 4 — relational database and administration
- Git — version control
- Physical or emulated Android device with GPS — field testing
- A laptop with at least 8 GB RAM recommended for Android emulator + PostgreSQL + Node concurrently

The implementation follows Form 3, Part B (Hierarchical/Iterative Design). Unlike the original Form 3 plan that separated a React web dashboard from the Flutter mobile client, this sprint delivers admin and manager capabilities inside the same Flutter application using role-based navigation, reducing deployment effort while preserving JWT security and per-rep data isolation in PostgreSQL.

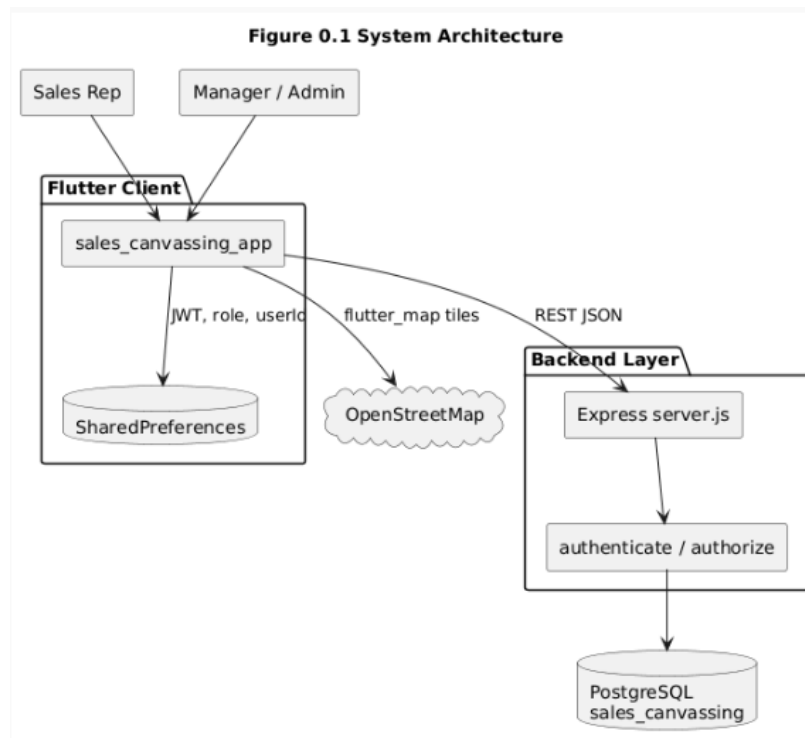


Figure 0.1 System Architecture Overview

1. DFD for User Authentication (Login & JWT)

Figure 1.1 DFD — User Authentication

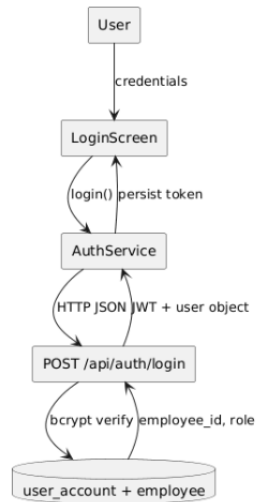


Figure 1.1 DFD for User Authentication

1.1 Login Screen

Figure 1.1.1 1.1 Login Screen

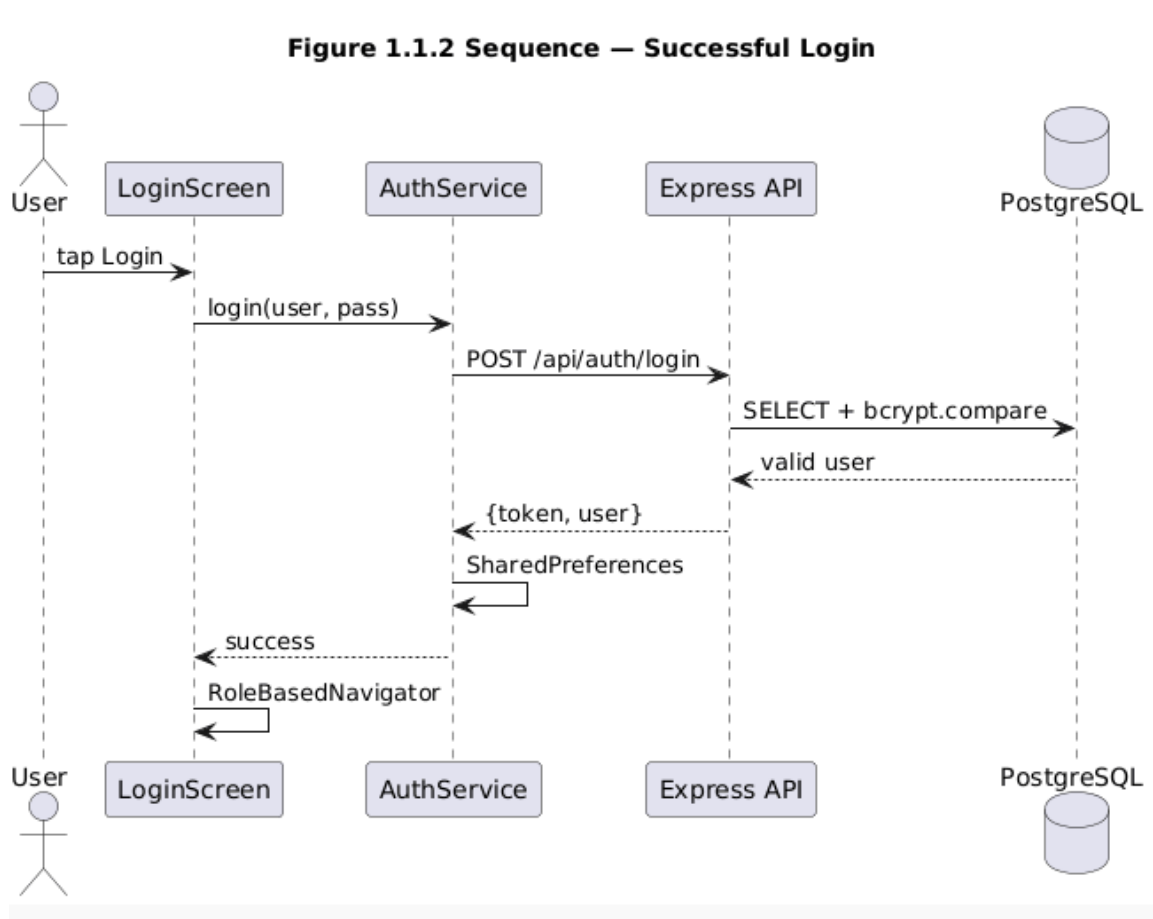


image for Figure 1.1.1

The authentication module is implemented with Node.js Express on the backend and PostgreSQL as the persistent store. The login endpoint `POST /api/auth/login` receives username and password from the mobile client, queries the `user_account` table joined with `employee`, and validates the password using `bcrypt.compare` against `password_hash`. If the account is active (`employee.is_active`), the server signs a JSON Web Token (JWT) with payload `user_id`, `employee_id`, `username`, and `role`, with expiration of 24 hours. The `last_login` column in `user_account` is updated to support audit trails.

On the Flutter side, the http package is used for communication between the mobile app and the Express API. The AuthService class encapsulates login, logout, and local session persistence using SharedPreferences keys auth_token, user_role, user_name, and user_id. ApiConfig resolves the API host dynamically: 10.0.2.2 for Android emulator, localhost for desktop, or API_HOST from dart-define for physical devices. This design aligns with Form 3 security requirements (JWT authentication and role-based access) while keeping configuration flexible for field testing.

The LoginScreen StatefulWidget implements _login() as an asynchronous handler. After validating non-empty username and password, it calls AuthService.login(). On success, Navigator.pushReplacement navigates to RoleBasedNavigator so the user cannot return to the login page with the back button. On failure, _errorMessage is displayed in red below the form. This follows the same navigation pattern described in comparable capstone implementations.

Listing 1.1.1 — Backend: authenticate() middleware (server.js)

```
function authenticate(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1];
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid token' });
  }
}

function authorize(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden: Insufficient permissions' });
    }
    next();
  };
}
```

```
function calculateDistance(lat1, lon1, lat2, lon2) {
  const R = 6371e3;
  const  $\phi_1$  = lat1 * Math.PI / 180;
  const  $\phi_2$  = lat2 * Math.PI / 180;
  const  $\Delta\phi$  = (lat2 - lat1) * Math.PI / 180;
  const  $\Delta\lambda$  = (lon2 - lon1) * Math.PI / 180;
  const a = Math.sin( $\Delta\phi/2$ ) * Math.sin( $\Delta\phi/2$ ) +
    Math.cos( $\phi_1$ ) * Math.cos( $\phi_2$ ) *
    Math.sin( $\Delta\lambda/2$ ) * Math.sin( $\Delta\lambda/2$ );
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));
  return R * c;
}
```

Listing 1.1.2 — Backend: login handler with bcrypt and JWT (server.js)

```
try {
  const result = await pool.query(
    `SELECT u.user_id, u.employee_id, u.username, u.role, u.password_hash, e.name, e.is_active
    FROM user_account u
    JOIN employee e ON e.employee_id = u.employee_id
    WHERE u.username = $1`,
    [username]
  );

  if (result.rows.length === 0) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const user = result.rows[0];

  if (user.is_active === false) {
    return res.status(401).json({ error: 'Account is deactivated' });
  }

  const passwordValid = await bcrypt.compare(password, user.password_hash);
  if (!passwordValid) {
    return res.status(401).json({
      error:
        'Username atau password salah. Setelah import SQL, jalankan: npm run setup-passwords',
    });
  }

  const token = jwt.sign(
    { user_id: user.user_id, employee_id: user.employee_id, username: user.username, role: user.role },
    JWT_SECRET,
    { expiresIn: '24h' }
  );

  await pool.query(`UPDATE user_account SET last_login = NOW() WHERE user_id = $1`, [user.user_id]);

  res.json({
    success: true,
    token,
    user: {
      id: user.employee_id,
      username: user.username,
      role: user.role,
      name: user.name
    }
  });
} catch (err) {
  console.error('Login error:', err);
  res.status(500).json({ error: 'Login failed' });
}
});
```

Listing 1.1.3 — Flutter: ApiConfig base URL (api_config.dart)

```
import 'dart:io' show Platform;

import 'package:flutter/foundation.dart' show kIsWeb;

class ApiConfig {
  static const String _envHost = String.fromEnvironment('API_HOST');

  static const int port = int.fromEnvironment(
    'API_PORT',
    defaultValue: 3000,
  );

  static String get host {
    if (_envHost.isNotEmpty) return _envHost;
    if (kIsWeb) return 'localhost';
    try {
      if (Platform.isAndroid) return '10.0.2.2';
    } catch (_) {}
    return 'localhost';
  }

  static String get baseUrl => 'http://$host:$port/api';
}
```

Listing 1.1.4 — Flutter: AuthService.login() (auth_service.dart)

```
static Future<Map<String, dynamic>> login(
  String username,
  String password,
) async {
  try {
    final response = await http.post(
      Uri.parse('$baseUrl/auth/login'),
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({'username': username, 'password': password}),
    );

    final data = jsonDecode(response.body);

    if (response.statusCode == 200 && data['success'] == true) {
      final user = Map<String, dynamic>.from(data['user'] as Map);
      final userId = _parseUserId(user);
      if (userId <= 0) {
        return {
          'success': false,
          'error': 'Login berhasil tetapi ID user tidak valid. Coba login ulang.',
        };
      }
    }
  }
}
```

```

        await saveToken(data['token']);
        await saveUserInfo(
          role: user['role']?.toString() ?? 'rep',
          name: user['name']?.toString() ?? 'User',
          userId: userId,
        );
        return {'success': true, 'user': user};
      } else {
        return {'success': false, 'error': data['error'] ?? 'Login failed'};
      }
    } catch (e) {
      return {
        'success': false,
        'error':
          'Tidak bisa hubungi server di $baseUrl. Jalankan backend (npm start) dan periksa IP/API_HOST.',
      };
    }
  }
}
}

```

Listing 1.1.5 — Flutter: LoginScreen._login() (login_screen.dart)

```

Future<void> _login() async {
  final String username = _usernameController.text.trim();
  final String password = _passwordController.text.trim();

  if (username.isEmpty || password.isEmpty) {
    setState(() {
      _errorMessage = 'Username and password required';
    });
    return;
  }

  setState(() {
    _isLoading = true;
    _errorMessage = null;
  });

  print('🔑 Attempting login with: $username');
  final result = await AuthService.login(username, password);

  if (result['success'] == true && mounted) {
    print('🔑 Login successful, navigating to MainNavigation');
    Navigator.pushReplacement(
      context,
      MaterialPageRoute(
        builder: (context) =>
          const RoleBasedNavigator(), // MaterialPageRoute
      );
  } else {
    print('🔑 Login failed: ${result['error']}');
    setState(() {
      _errorMessage = result['error'] ?? 'Login failed';
      _isLoading = false;
    });
  }
}

```

```

    }
  }

  @override
  void dispose() {
    _usernameController.dispose();
    _passwordController.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        decoration: const BoxDecoration(
          gradient: LinearGradient(
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
            colors: [Colors.blue, Colors.purple],
          ), // LinearGradient
        ), // BoxDecoration
      ),
    );
  }
}

```

Figure 1.1.3 Login Screen

SalesCanvas branding, username/password, Login button.

[Insert application screenshot — Login Screen]

Figure 1.1.4 Login Failed

Error message for wrong password or network failure.

[Insert application screenshot — Login Failed]

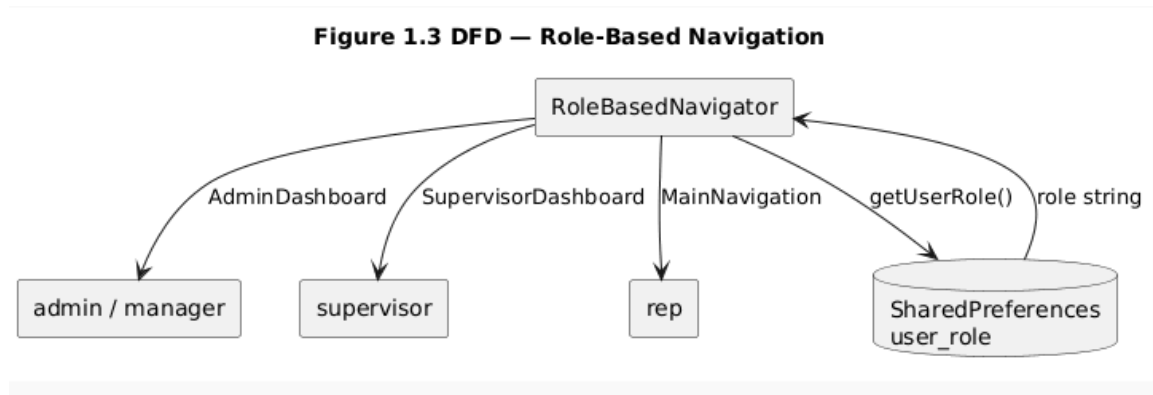
Figure 1.1.5 Login Success

Transition to role-specific home screen.

[Insert application screenshot — Login Success]

1.2 Role-Based Navigation After Login

Figure 1.2 1.2 Role-Based Navigation After Login



[image for Figure 1.2]

After authentication, RoleBasedNavigator reads the stored role from SharedPreferences via AuthService.getUserRole(). Administrators and managers are routed to AdminDashboard for master data and sales team management. Supervisors see SupervisorDashboard with team-level KPIs from GET /api/supervisor/team-dashboard. Sales representatives enter MainNavigation with a drawer containing Dashboard, Visits, Route Planning, GPS Tracking, Order Taking, Stock, Collections, Analytics, Merchandising, Outlet Management, and Settings.

The authorize (roles) middleware on protected routes enforces server-side RBAC. For example, GET /api/admin/sales-reps requires admin or manager role; a rep receives HTTP 403 Forbidden if attempting admin endpoints. Combined with JWT employee_id filtering, this implements defense in depth for per-user data isolation required by the project.

Listing 1.2.1 — Flutter: RoleBasedNavigator (main.dart)

```

class RoleBasedNavigator extends StatelessWidget {
  const RoleBasedNavigator({super.key});

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<String>(
      future: AuthService.getUserRole(),
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Scaffold(
            body: Center(child: CircularProgressIndicator()),
          ); // Scaffold
        }
        final role = snapshot.data ?? 'rep';
        if (role == 'admin' || role == 'manager') {
          return const AdminDashboard();
        }
        if (role == 'supervisor') {
          return const SupervisorDashboard();
        }
        return const MainNavigation();
      },
    ); // FutureBuilder
  }
}

class MainNavigation extends StatefulWidget {
  const MainNavigation({super.key});

  @override
  State<MainNavigation> createState() => _MainNavigationState();
}

```

Listing 1.2.2 — Backend: authorize middleware (server.js)

```

function authorize(...roles) {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Forbidden: Insufficient permissions' });
    }
    next();
  };
}

```

Listing 1.2.3 — Flutter: ApiClient JWT headers (api_client.dart)

```
static Future<Map<String, dynamic>> postMap(  
  String endpoint,  
  Map<String, dynamic> body,  
) async {  
  return asMap(await post(endpoint, body));  
}  
  
static Future<dynamic> put(  
  String endpoint,  
  Map<String, dynamic> body,  
) async {  
  try {  
    final headers = await _getHeaders();  
    final url = Uri.parse('$baseUrl$endpoint');  
  
    final response = await http.put(  
      url,  
      headers: headers,  
      body: jsonEncode(body),  
    );  
  
    if (response.statusCode == 200) {  
      return jsonDecode(response.body);  
    } else {  
      throw Exception('HTTP ${response.statusCode}');  
    }  
  } catch (e) {  
    throw Exception('Network error: $e');  
  }  
}
```

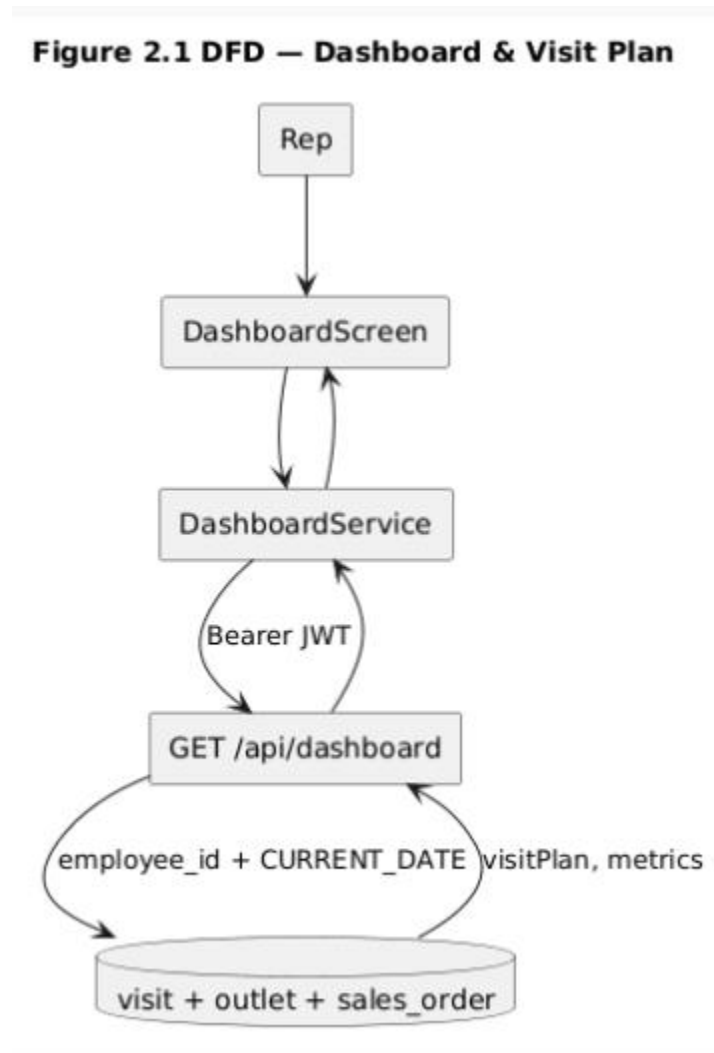
Figure 1.2.1 Admin Dashboard

List of sales reps for manager role.

[Insert application screenshot — Admin Dashboard]

2. DFD for Dashboard, Visit Plan & Visits Management

Figure 2.1 DFD Dashboard and Visits



[image for Figure 2.1]

2.1 Dashboard Screen

The dashboard is the primary operational view for sales representatives at the start of a canvassing day. `DashboardService.fetchDashboard()` invokes `GET /api/dashboard` with the Bearer token. The backend extracts `req.user.employee_id` from the decoded JWT and, for role rep, returns only visits where `visit_date` equals `CURRENT_DATE` and status is `Planned` or `InProgress`, joined with outlet details (name, address, priority, GPS). Metrics include `totalSalesToday` (sum of order totals for today's visits), `strikeRate` (percentage of visits with at least one order), and `pendingOrders`.

The `DashboardData` model class structures the JSON response using factory constructors `VisitPlanItem.fromJson` and `Metrics.fromJson`, with defensive parsing for numeric types returned as strings from PostgreSQL. `DashboardScreen` uses `FutureBuilder` and `RefreshIndicator` so the rep can pull-to-refresh the plan after manager assignment changes.

Listing 2.1.1 — Flutter: DashboardService

```
import 'api_client.dart';
import '../models/dashboard_data.dart';

class DashboardService {
  static Future<DashboardData> fetchDashboard() async {
    try {
      print('📄 DashboardService: Fetching dashboard...');
      final data = await ApiClient.get('/dashboard');
      print('📄 DashboardService: Data received successfully');
      return DashboardData.fromJson(data);
    } catch (e) {
      print('❌ DashboardService error: $e');
      throw Exception('Failed to load dashboard: $e');
    }
  }
}
```

Listing 2.1.2 — Flutter: DashboardData model (excerpt)

```
class DashboardData {
  final List<VisitPlanItem> visitPlan;
  final Metrics metrics;
  final DateTime lastSync;

  DashboardData({
    required this.visitPlan,
    required this.metrics,
    required this.lastSync,
  });

  factory DashboardData.fromJson(Map<String, dynamic> json) {
    final list = json['visitPlan'];
    final List<VisitPlanItem> planItems = list is List
      ? list
        .map((i) => VisitPlanItem.fromJson(
          Map<String, dynamic>.from(i as Map),
        )) // VisitPlanItem.fromJson
        .toList()
      : [];

    return DashboardData(
      visitPlan: planItems,
      metrics: Metrics.fromJson(
        Map<String, dynamic>.from(json['metrics'] as Map? ?? {}),
      ), // Metrics.fromJson
      lastSync: DateTime.tryParse(json['lastSync']?.toString() ?? '') ??
        DateTime.now(),
    ); // DashboardData
  }
}

class VisitPlanItem {
  final int outletId;
  final String outletName;
  final String address;
  final String priority;
  final String visitTime;
  final String status;
  final double latitude;
  final double longitude;
}
```

Listing 2.1.3 — Backend: dashboard for rep (server.js)

```
app.get('/api/dashboard', authenticate, async (req, res) => {
  const employeeId = req.user.employee_id;
  const role = req.user.role;

  if (!pool) {
    return res.json({
      success: true,
      visitPlan: [
        { outlet_id: 1, outlet_name: "Toko Maju Jaya", address: "Jl. Merdeka No. 123", priority: "A", visit_time: "0" },
      ],
      metrics: { totalSalesToday: 330000, strikeRate: 50.0, pendingOrders: 0 },
      lastSync: new Date().toISOString()
    });
  }

  try {
    let visitPlanQuery, metricsQuery;
    let params = [employeeId];

    if (role === 'rep') {
      visitPlanQuery = `
        SELECT o.outlet_id, o.outlet_name, o.address, o.priority, v.visit_time, v.status, o.latitude, o.longitude
        FROM outlet o
        JOIN visit v ON v.outlet_id = o.outlet_id
        WHERE v.employee_id = $1
        AND v.visit_date = CURRENT_DATE
        AND v.status IN ('Planned', 'InProgress')
        ORDER BY v.visit_time
      `;
      metricsQuery = `
        SELECT COALESCE(SUM(so.total), 0) as total_sales_today,
               COUNT(DISTINCT v.visit_id) as total_visits,
               COUNT(DISTINCT CASE WHEN so.order_id IS NOT NULL THEN v.visit_id END) as productive_visits
        FROM visit v
        LEFT JOIN sales_order so ON so.visit_id = v.visit_id
        WHERE v.employee_id = $1 AND v.visit_date = CURRENT_DATE
        AND v.status IN ('Planned', 'InProgress', 'Completed')
      `;
    } else {
      params = [];
      visitPlanQuery = `
        SELECT o.outlet_id, o.outlet_name, o.address, o.priority, v.visit_time, v.status, o.latitude, o.longitude, e
        FROM outlet o
        JOIN visit v ON v.outlet_id = o.outlet_id
        JOIN employee e ON e.employee_id = v.employee_id
        WHERE v.visit_date = CURRENT_DATE
        AND v.status IN ('Planned', 'InProgress')
        ORDER BY v.visit_time
      `;
      metricsQuery = `
        SELECT COALESCE(SUM(so.total), 0) as total_sales_today,
               COUNT(DISTINCT v.visit_id) as total_visits,
               COUNT(DISTINCT CASE WHEN so.order_id IS NOT NULL THEN v.visit_id END) as productive_visits
        FROM visit v
        LEFT JOIN sales_order so ON so.visit_id = v.visit_id
        WHERE v.visit_date = CURRENT_DATE
        AND v.status IN ('Planned', 'InProgress', 'Completed')
      `;
    }

    const [visitPlanResult, metricsResult] = await Promise.all([
      pool.query(visitPlanQuery, params),
      pool.query(metricsQuery, params)
    ]);

    const metrics = metricsResult.rows[0];
    const strikeRate = metrics.total_visits > 0
```

```

const [visitPlanResult, metricsResult] = await Promise.all([
  pool.query(visitPlanQuery, params),
  pool.query(metricsQuery, params)
]);

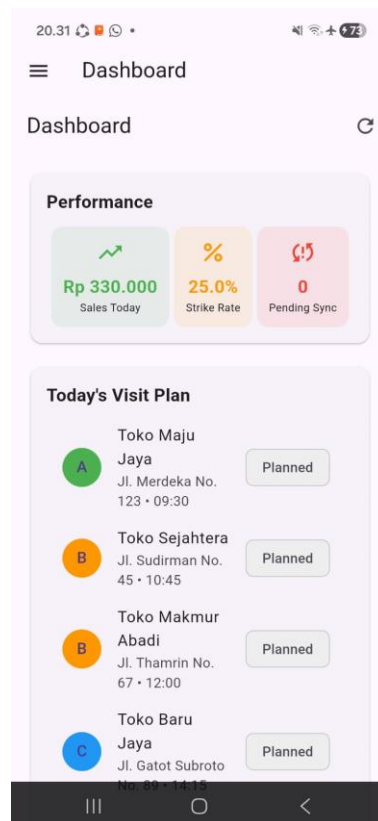
const metrics = metricsResult.rows[0];
const strikeRate = metrics.total_visits > 0
  ? (metrics.productive_visits / metrics.total_visits * 100).toFixed(1)
  : 0;

res.json({
  success: true,
  visitPlan: visitPlanResult.rows,
  metrics: {
    totalSalesToday: parseFloat(metrics.total_sales_today),
    strikeRate: parseFloat(strikeRate),
    pendingOrders: 0,
  },
  lastSync: new Date().toISOString(),
});
} catch (err) {
  console.error('Dashboard error:', err);
  res.status(500).json({ error: err.message });
}
});

```

Figure 2.1.1 Rep Dashboard

Visit plan cards and KPI summary.



[Insert application screenshot — Rep Dashboard]

2.2 Visits Screen — Today List & Missed Visit

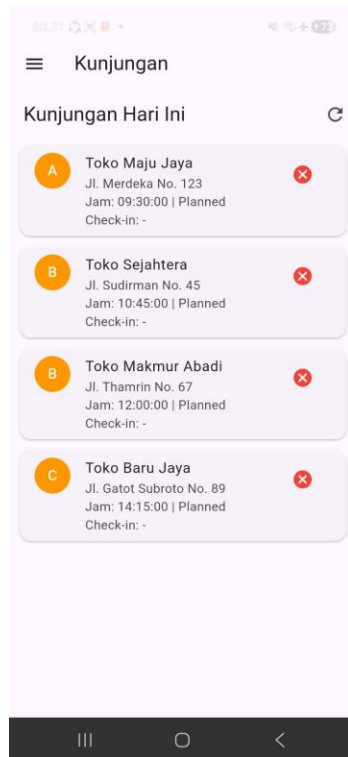
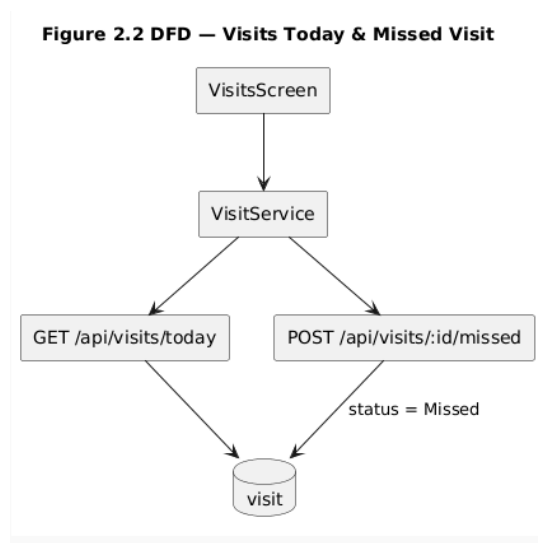


Figure 2.2 2.2 Visits Screen — Today List & Missed Visit



[Insert rendered image for Figure 2.2]

VisitsScreen displays all visits for the current day via VisitService.fetchTodayVisits(), which calls GET /api/visits/today. Each row shows outlet name, scheduled visit_time, status (Planned, InProgress, Completed, Missed), and check-in/check-out timestamps when available.

When a rep cannot attend an outlet, markMissed posts to /api/visits/:id/missed with a reason string. The backend updates visit.status to Missed and stores visit_reason, supporting Form 3 missed visit tracking and supervisor reporting. Reschedule is available for admin/manager/rep via POST /api/visits/:id/reschedule.

Listing 2.2.1 — Flutter: VisitService

```
import 'api_client.dart';

class VisitService {
  static Future<List<Map<String, dynamic>>> fetchTodayVisits({int? repId}) async {
    final endpoint =
      repId != null ? '/visits/today?rep_id=$repId' : '/visits/today';
    final data = await ApiClient.get(endpoint);
    if (data is List) {
      return List<Map<String, dynamic>>.from(data);
    }
    return [];
  }

  static Future<void> markMissed(int visitId, String reason) async {
    await ApiClient.post('/visits/$visitId/missed', {'reason': reason});
  }

  static Future<void> reschedule(
    int visitId, {
    required String visitDate,
    String? visitTime,
  }) async {
    await ApiClient.post('/visits/$visitId/reschedule', {
      'visit_date': visitDate,
      if (visitTime != null) 'visit_time': visitTime,
    });
  }
}
```

Listing 2.2.2 — Backend: visits today query (server.js)

```

try {
  let query;
  let params;

  if (role === 'rep') {
    query = `
      SELECT v.visit_id, v.outlet_id, o.outlet_name, o.address, v.visit_time, v.status,
      | | | | v.check_in_time, v.check_out_time, v.visit_reason, o.priority
      FROM visit v
      JOIN outlet o ON o.outlet_id = v.outlet_id
      WHERE v.employee_id = $1 AND v.visit_date = CURRENT_DATE
      ORDER BY v.visit_time
    `;
    params = [employeeId];
  } else {
    query = `
      SELECT v.visit_id, v.outlet_id, o.outlet_name, o.address, v.visit_time, v.status,
      | | | | v.check_in_time, v.check_out_time, v.visit_reason, o.priority, e.name as sales_name
      FROM visit v
      JOIN outlet o ON o.outlet_id = v.outlet_id
      JOIN employee e ON e.employee_id = v.employee_id
      WHERE v.employee_id = $1 AND v.visit_date = CURRENT_DATE
      ORDER BY v.visit_time
    `;
    params = [repId];
  }

  const result = await pool.query(query, params);
  res.json(result.rows);
} catch (err) {}
res.status(500).json({ error: err.message });
});

```

Listing 2.2.3 — Backend: mark missed (server.js)

```

}

try {
  const result = await pool.query(
    `UPDATE visit SET status = 'Missed', visit_reason = $1, notes = $2
     WHERE visit_id = $3 AND employee_id = $4
     RETURNING visit_id`,
    [reason || 'Missed', reason, visitId, employeeId]
  );
  if (result.rows.length === 0) {
    return res.status(404).json({ error: 'Visit not found' });
  }
  res.json({ success: true, message: 'Visit marked as missed' });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

app.post('/api/visits/:id/reschedule', authenticate, authorize('admin', 'manager', 'rep'), async (req, res) => {
  const { visit_date, visit_time } = req.body;
  const visitId = req.params.id;

  if (!visit_date) {
    return res.status(400).json({ error: 'visit_date required' });
  }

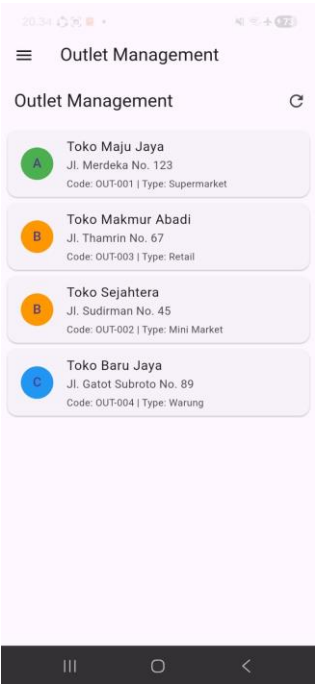
  if (!pool) {
    return res.json({ success: true });
  }

  try {
    let result;
    if (req.user.role === 'rep') {

```

Figure 2.2.1 Visits Screen

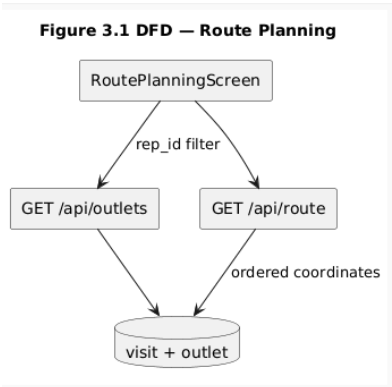
List of today's visits with status badges.



[Insert application screenshot — Visits Screen]

3. DFD for Route Planning & GPS Tracking

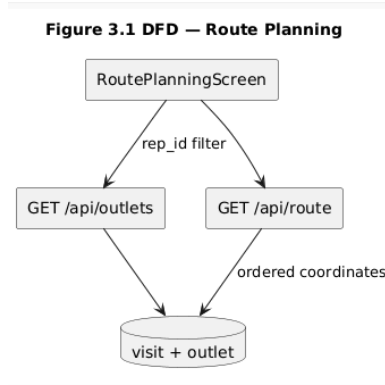
Figure 3.1 DFD Route and GPS



[Insert rendered image for Figure 3.1]

3.1 Route Planning Screen

Figure 3.1.1 3.1 Route Planning Screen



[Insert rendered image for Figure 3.1.1]

RoutePlanningScreen visualizes the rep's assigned outlets on an interactive map using `flutter_map` and `OpenStreetMap` tile layers, as specified in Form 3. Outlet markers are loaded from `GET /api/outlets?rep_id={employeeId}` so only outlets linked to active Planned/InProgress visits for that rep are shown. `GET /api/route` returns an ordered list of latitude/longitude pairs sorted by `visit_time` for simple route visualization (full TSP optimization is planned for a later sprint).

Listing 3.1.1 — Backend: outlets filter by rep (server.js)

```
app.get('/api/outlets', authenticate, async (req, res) => {
  const role = req.user.role;
  let repId = req.query.rep_id;
  if (role === 'rep') {
    repId = String(req.user.employee_id);
  }

  if (!pool) {
    let mockOutlets = [
      { outlet_id: 1, outlet_name: "Toko Maju Jaya", address: "Jl. Merdeka No. 123", latitude: -6.2088, longitude: 100.5795 },
      { outlet_id: 2, outlet_name: "Toko Sejahtera", address: "Jl. Sudirman No. 45", latitude: -6.2095, longitude: 100.5805 },
      { outlet_id: 3, outlet_name: "Toko Makmur Abadi", address: "Jl. Thamrin No. 67", latitude: -6.2078, longitude: 100.5788 },
      { outlet_id: 4, outlet_name: "Toko Baru Jaya", address: "Jl. Gatot Subroto No. 89", latitude: -6.2102, longitude: 100.5812 }
    ];
  }

  if (repId) {
    const repIdNum = parseInt(repId);
    if (repIdNum === 3) {
      mockOutlets = mockOutlets.filter(o => o.outlet_id === 1 || o.outlet_id === 2);
    } else if (repIdNum === 4) {
      mockOutlets = mockOutlets.filter(o => o.outlet_id === 3 || o.outlet_id === 4);
    }
  }

  return res.json(mockOutlets);
})

try {
  let query = `
    SELECT o.outlet_id, o.outlet_name, o.address, o.latitude, o.longitude, o.priority, o.store_type, o.outlet_code
    FROM outlet o
    WHERE o.is_active = true
  `;
  let params = [];
}
```

```

    if (repId) {
      query += ` AND o.outlet_id IN (
        SELECT DISTINCT v.outlet_id
        FROM visit v
        WHERE v.employee_id = $1
        AND v.status IN ('Planned', 'InProgress')
      )`;
      params.push(repId);
    }

    query += ` ORDER BY o.priority ASC, o.outlet_name ASC`;

    const result = await pool.query(query, params);
    res.json(result.rows);
  } catch (err) {
    console.error('Error fetching outlets:', err);
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/outlets', authenticate, authorize('admin', 'manager'), async (req, res) => {
  const { outlet_code, outlet_name, address, latitude, longitude, owner_name, phone, credit_limit, priority, store_t } = req.body;
  if (!outlet_code || !outlet_name || !address) {
    return res.status(400).json({ error: 'Required fields missing' });
  }
  try {
    const result = await pool.query(
      `INSERT INTO outlet (outlet_code, outlet_name, address, latitude, longitude, owner_name, phone, credit_limit,
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11) RETURNING outlet_id`,
      [outlet_code, outlet_name, address, latitude, longitude, owner_name, phone, credit_limit || 0, priority || 'C'
    );
    res.status(201).json({ success: true, outlet_id: result.rows[0].outlet_id });
  } catch (err) {

```

Listing 3.1.2 — Backend: route coordinates (server.js)

```

app.get('/api/route', authenticate, async (req, res) => {
  const employeeId = req.user.employee_id;
  const role = req.user.role;

  if (!pool) return res.json([ { latitude: -6.2088, longitude: 106.8456 } ]);

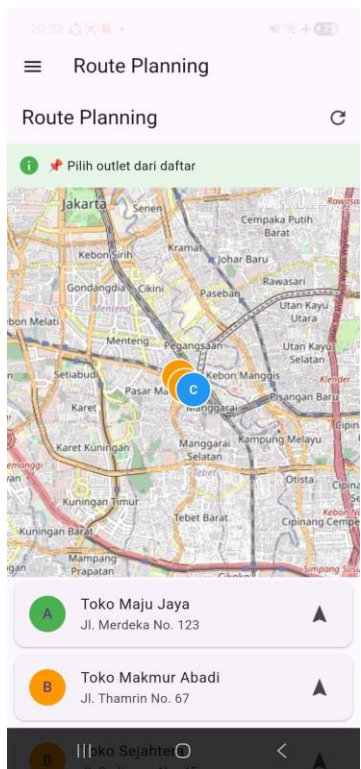
  try {
    let query;
    let params = [];

    if (role === 'rep') {
      query = `
        SELECT o.latitude, o.longitude
        FROM outlet o
        INNER JOIN visit v ON v.outlet_id = o.outlet_id
        WHERE v.employee_id = $1
        AND v.visit_date >= CURRENT_DATE
        AND v.status IN ('Planned', 'InProgress')
        AND o.latitude IS NOT NULL
        ORDER BY v.visit_time
      `;
      params = [employeeId];
    } else {
      query = `
        SELECT latitude, longitude FROM outlet
        WHERE is_active = true AND latitude IS NOT NULL
        ORDER BY outlet_id
      `;
    }

```

Figure 3.1.1 Route Planning Map

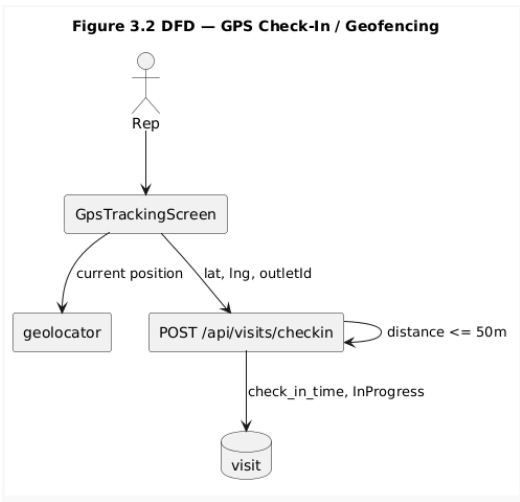
Markers for priority A/B/C outlets.



[Insert application screenshot — Route Planning Map]

3.2 GPS Tracking — Check-In and Check-Out

Figure 3.2 3.2 GPS Tracking — Check-In and Check-Out



[Insert rendered image for Figure 3.2]

GpsTrackingScreen requests location permission and reads the device position through the geolocator package. When the rep taps Check-in, the app sends outletId, latitude, and longitude to POST /api/visits/checkin. The server loads outlet coordinates and computes Haversine distance in metres using calculateDistance(). If distance exceeds 50 metres, the API returns HTTP 400 with error Outside geofence — implementing Form 3 geo-fencing validation.

If a visit row already exists for the rep, outlet, and CURRENT_DATE, check-in updates check_in_time, check_in_lat/lng, and sets status InProgress. Otherwise a new visit row is inserted. Check-out via POST /api/visits/checkout sets check_out_time and status Completed, which is required before meaningful end-of-visit reporting.

Listing 3.2.1 — Backend: Haversine helper (server.js)

```
function calculateDistance(lat1, lon1, lat2, lon2) {  
  const R = 6371e3;  
  const  $\phi_1$  = lat1 * Math.PI / 180;  
  const  $\phi_2$  = lat2 * Math.PI / 180;  
  const  $\Delta\phi$  = (lat2 - lat1) * Math.PI / 180;  
  const  $\Delta\lambda$  = (lon2 - lon1) * Math.PI / 180;  
  const a = Math.sin( $\Delta\phi$ /2) * Math.sin( $\Delta\phi$ /2) +  
    Math.cos( $\phi_1$ ) * Math.cos( $\phi_2$ ) *  
    Math.sin( $\Delta\lambda$ /2) * Math.sin( $\Delta\lambda$ /2);  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a));  
  return R * c;  
}
```

Listing 3.2.2 — Backend: check-in handler (server.js)

```

app.post('/api/visits/checkin', authenticate, async (req, res) => {
  const { outletId, latitude, longitude, visitReason } = req.body;
  const employeeId = req.user.employee_id;

  if (!outletId || !latitude || !longitude) {
    return res.status(400).json({ error: 'Incomplete data' });
  }

  if (!pool) {
    return res.json({ success: true, message: 'Check-in successful (mock)', visitId: 999, distance: 5.2, outletName: '' });
  }

  try {
    const outletResult = await pool.query(`SELECT latitude, longitude, outlet_name FROM outlet WHERE outlet_id = $1`);
    if (outletResult.rows.length === 0) return res.status(404).json({ error: 'Outlet not found' });

    const outlet = outletResult.rows[0];
    const distance = calculateDistance(latitude, longitude, parseFloat(outlet.latitude), parseFloat(outlet.longitude));
    if (distance > 50) {
      return res.status(400).json({ error: 'Outside geofence', distance, required: 50 });
    }

    const existingVisit = await pool.query(
      `SELECT visit_id FROM visit WHERE employee_id = $1 AND outlet_id = $2 AND visit_date = CURRENT_DATE`,
      [employeeId, outletId]
    );

    let visitId;
    if (existingVisit.rows.length > 0) {
      visitId = existingVisit.rows[0].visit_id;
      await pool.query(
        `UPDATE visit SET check_in_time = CURRENT_TIMESTAMP, check_in_lat = $1, check_in_lng = $2, visit_reason = $3`
        [latitude, longitude, visitReason, visitId]
      );
    } else {
      const result = await pool.query(
        `INSERT INTO visit (employee_id, outlet_id, visit_date, check_in_time, check_in_lat, check_in_lng, visit_reason)`
        `VALUES ($1, $2, CURRENT_DATE, CURRENT_TIMESTAMP, $3, $4, $5, 'InProgress') RETURNING visit_id`,
        [employeeId, outletId, latitude, longitude, visitReason]
      );
      visitId = result.rows[0].visit_id;
    }

    res.json({ success: true, message: 'Check-in successful', visitId, distance: distance.toFixed(1), outletName: outlet.outlet_name });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.post('/api/visits/checkout', authenticate, async (req, res) => {
  const { visitId } = req.body;
  const employeeId = req.user.employee_id;

  if (!visitId) return res.status(400).json({ error: 'Visit ID required' });

  if (!pool) return res.json({ success: true, message: 'Check-out successful (mock)' });

  try {
    const result = await pool.query(
      `UPDATE visit SET check_out_time = CURRENT_TIMESTAMP, status = 'Completed' WHERE visit_id = $1 AND employee_id = $2`
      [visitId, employeeId]
    );
    if (result.rows.length === 0) return res.status(404).json({ error: 'Visit not found' });
    res.json({ success: true, message: 'Check-out successful' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

Listing 3.2.3 — Backend: check-out handler (server.js)

```
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  });

app.post('/api/visits/checkout', authenticate, async (req, res) => {
  const { visitId } = req.body;
  const employeeId = req.user.employee_id;

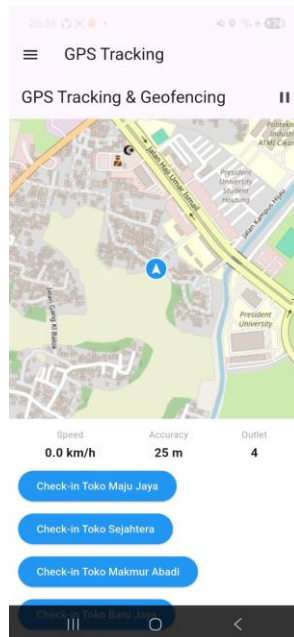
  if (!visitId) return res.status(400).json({ error: 'Visit ID required' });

  if (!pool) return res.json({ success: true, message: 'Check-out successful (mock)' });

  try {
    const result = await pool.query(
      `UPDATE visit SET check_out_time = CURRENT_TIMESTAMP, status = 'Completed' WHERE visit_id = $1 AND employee_id [visitId, employeeId]`
    );
    if (result.rows.length === 0) return res.status(404).json({ error: 'Visit not found' });
    res.json({ success: true, message: 'Check-out successful' });
  } catch (err) {}
  res.status(500).json({ error: err.message });
});
```

Figure 3.2.1 GPS Tracking Screen

Map, distance indicator, Check-in/Check-out.

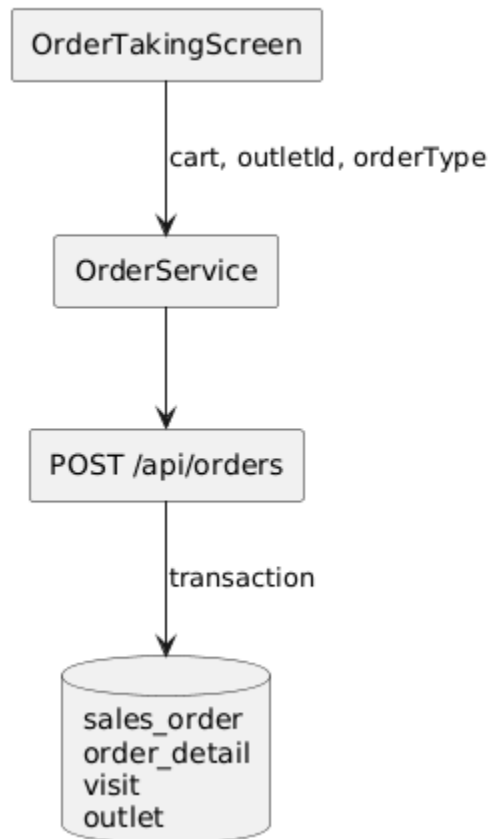


[Insert application screenshot — GPS Tracking Screen]

4. DFD for Order Taking

Figure 4.1 DFD Order Taking

Figure 4.1 DFD — Order Taking



[Insert rendered image for Figure 4.1]

4.1 Order Taking Screen & OrderService

OrderTakingScreen is the core revenue capture module. On init, `_loadData()` fetches products via `OrderService.fetchProducts()` (GET /api/products), outlets assigned to the logged-in rep, and active promotions via `PromotionService.fetchActive()`. The user builds a cart of `CartItem` objects,

selects order type (Sales, Pre-order, Return, FOC per Form 3), payment method, and optional promotion.

`OrderService.submitOrder()` posts a JSON body to `/api/orders` containing `outletId`, `items[{productId, quantity, price}]`, `orderType`, and optional `promotionId`. The backend runs a database transaction: validates outlet credit limit for Sales/Pre-order, ensures check-in exists for Sales/Return, inserts `sales_order` and `order_detail` rows, updates `outlet.outstanding`, and may transition visit status from Planned to InProgress.

Listing 4.1.1 — Flutter: OrderService

```
class OrderService {
  static Future<List<Product>> fetchProducts() async {
    final data = await ApiClient.get('/products');
    if (data is List) {
      return data.map((json) => Product.fromJson(json)).toList();
    }
    return [];
  }

  static Future<Map<String, dynamic>> submitOrder({
    required int outletId,
    required List<CartItem> items,
    required double total,
    required String paymentMethod,
    required String orderType,
    int? promotionId,
  }) async {
    return ApiClient.postMap('/orders', {
      'outletId': outletId,
      'orderType': orderType,
      if (promotionId != null) 'promotionId': promotionId,
      'items': items
        .map(
          (item) => {
            'productId': item.product.productId,
            'quantity': item.quantity,
            'price': item.product.price,
          },
        )
        .toList(),
      'total': total,
      'paymentMethod': paymentMethod,
    });
  }
}
```

Listing 4.1.2 — Flutter: Product.fromJson (product.dart)

```
factory Product.fromJson(Map<String, dynamic> json) {  
  return Product(  
    productId: _toInt(json['product_id']),  
    productCode: _toString(json['product_code']),  
    productName: _toString(json['product_name']),  
    price: _toDouble(json['price']),  
    stock: _toInt(json['stock']),  
    unitName: _toString(json['unit_name']),  
  );  
}  
  
static int _toInt(dynamic value) {  
  if (value == null) return 0;  
  if (value is int) return value;  
  if (value is double) return value.toInt();  
  if (value is String) return int.tryParse(value) ?? 0;  
  return 0;  
}  
  
static double _toDouble(dynamic value) {  
  if (value == null) return 0.0;  
  if (value is double) return value;  
  if (value is int) return value.toDouble();  
  if (value is String) return double.tryParse(value) ?? 0.0;  
  return 0.0;  
}  
  
static String _toString(dynamic value) {  
  if (value == null) return '';  
  return value.toString();  
}  
}
```

Listing 4.1.3 — Backend: order transaction start (server.js)

```

const validTypes = ['Sales', 'Pre-order', 'Return', 'FOC'];
const type = validTypes.includes(orderType) ? orderType : 'Sales';

if (!outletId || !items || items.length === 0) {
  return res.status(400).json({ error: 'Data tidak lengkap' });
}

if (!pool) {
  return res.status(201).json({ success: true, orderNumber: `SO-MOCK-${Date.now()}` });
}

const client = await pool.connect();
try {
  await client.query('BEGIN');

  const outletResult = await client.query(
    `SELECT outlet_id, credit_limit, outstanding, outlet_name FROM outlet WHERE outlet_id = $1 AND is_active = true`
    [outletId]
  );
  if (outletResult.rows.length === 0) {
    await client.query('ROLLBACK');
    return res.status(404).json({ error: 'Outlet not found' });
  }
  const outlet = outletResult.rows[0];

  let visitResult = await client.query(
    `SELECT visit_id, check_in_time, status FROM visit
    WHERE employee_id = $1 AND outlet_id = $2 AND visit_date = CURRENT_DATE`,
    [employeeId, outletId]
  );

  let visitId;
  if (visitResult.rows.length === 0) {

```

```

    if (type === 'Sales' || type === 'Return') {
      await client.query('ROLLBACK');
      return res.status(400).json({ error: 'Check-in di outlet ini diperlukan sebelum order' });
    }
    const newVisit = await client.query(
      `INSERT INTO visit (employee_id, outlet_id, visit_date, status)
      VALUES ($1, $2, CURRENT_DATE, 'Planned') RETURNING visit_id`,
      [employeeId, outletId]
    );
    visitId = newVisit.rows[0].visit_id;
  } else {
    visitId = visitResult.rows[0].visit_id;
    if ((type === 'Sales' || type === 'Return') && !visitResult.rows[0].check_in_time) {
      await client.query('ROLLBACK');
      return res.status(400).json({ error: 'Check-in diperlukan sebelum order Sales/Return' });
    }
  }
}

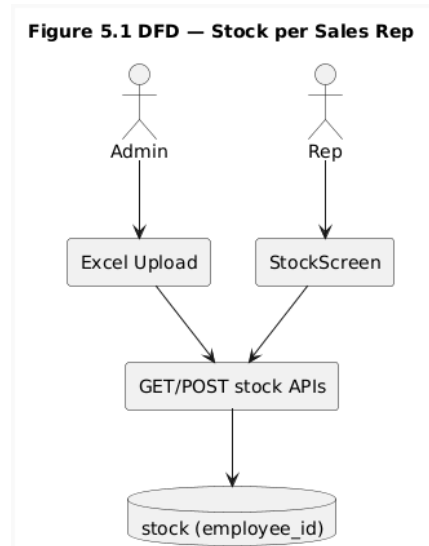
const orderNumber = `SO-${Date.now()}`;
let subtotal = items.reduce((sum, item) => sum + (item.price * item.quantity), 0);
if (type === 'Return') {
  subtotal = Math.abs(subtotal);
}

let discount = 0;
if (type === 'FOC') {
  subtotal = items.reduce((sum, item) => sum + (item.price * item.quantity), 0);
  discount = subtotal;
} else if (promotionId) {
  const promoResult = await client.query(
    `SELECT type, conditions, reward FROM promotion
    WHERE promotion_id = $1 AND is_active = true
    AND CURRENT_DATE BETWEEN start_date AND end_date`,

```

5. DFD for Stock Management

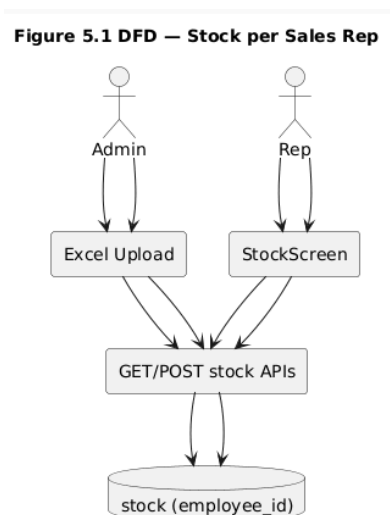
Figure 5.1 DFD Stock Management



[Insert rendered image for Figure 5.1]

5.1 Stock Screen (Per Sales Rep)

Figure 5.1.1 5.1 Stock Screen (Per Sales Rep)



[Insert rendered image for Figure 5.1.1]

Stock visibility is isolated per sales representative using the stock table composite key (product_id, type, employee_id) where type is distributor, van, or outlet. This satisfies the project requirement that each user sees different stock data. StockScreen calls GET /api/stock without rep_id when the logged-in user is a rep; the server automatically uses req.user.employee_id in subqueries.

Administrators open RepDetailScreen for a selected rep and upload Excel files through POST /api/admin/sales-reps/:id/stock/upload-excel. The multer middleware accepts .xlsx buffers; the xlsx library parses rows and upserts stock quantities. The log_stock_change trigger writes stock_history whenever quantity changes, providing an audit trail required for inventory governance.

Listing 5.1.1 — SQL: stock table definition

```
CREATE TABLE stock (  
  stock_id SERIAL PRIMARY KEY,  
  product_id INTEGER NOT NULL REFERENCES product(product_id),  
  employee_id INTEGER NOT NULL REFERENCES employee(employee_id),  
  type VARCHAR(20) NOT NULL CHECK (type IN ('distributor', 'van', 'outlet')),  
  quantity INTEGER NOT NULL DEFAULT 0,  
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_by INTEGER REFERENCES employee(employee_id),  
  UNIQUE(product_id, type, employee_id)  
);
```

Listing 5.1.2 — Backend: GET stock for rep (server.js)

```

        COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'outlet' AND employee_id = $
        CASE
            WHEN COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'distributor' AND emp
            WHEN COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'van' AND employee_id
            ELSE 'Good'
        END as status
        FROM product p WHERE p.is_active = true ORDER BY p.product_name
    `;
    params = [repId];
  } else if (role === 'rep') {
    query = `
      SELECT p.product_id, p.product_name, p.product_code, COALESCE(p.min_stock, 0) as min_stock,
      COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'distributor' AND employee_id
      COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'van' AND employee_id = $1),
      COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'outlet' AND employee_id = $
      CASE
          WHEN COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'distributor' AND emp
          WHEN COALESCE((SELECT quantity FROM stock WHERE product_id = p.product_id AND type = 'van' AND employee_id
          ELSE 'Good'
      END as status
      FROM product p WHERE p.is_active = true ORDER BY p.product_name
    `;
    params = [employeeId];
  } else {
    return res.status(400).json({
      error: 'Admin/manager harus menyertakan rep_id untuk melihat stok per sales rep',
    });
  }

  const result = await pool.query(query, params);
  res.json(result.rows);
} catch (err) {

```

Listing 5.1.3 — Flutter: StockService.fetchStock

```

import 'api_client.dart';
import '../models/stock_item.dart';

class StockService {
  static Future<List<StockItem>> fetchStock() async {
    try {
      final data = await ApiClient.get('/stock');
      print('📦 Stock data type: ${data.runtimeType}');

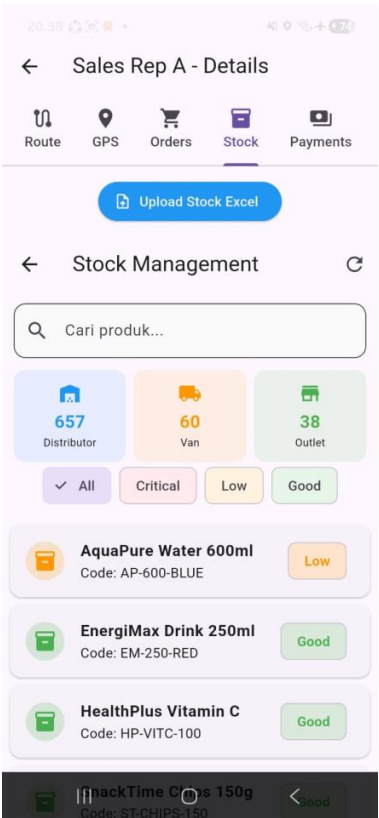
      if (data is List) {
        return data.map((json) => StockItem.fromJson(json)).toList();
      } else {
        print('❌ Stock data is not a List');
        return [];
      }
    } catch (e) {
      print('❌ fetchStock error: $e');
      throw Exception('Failed to load stock: $e');
    }
  }

  static Future<List<StockItem>> refreshStock() async {
    return fetchStock();
  }
}

```

Figure 5.1.1 Stock Screen

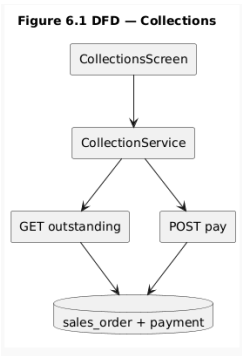
Table with distributor/van/outlet columns and status.



[Insert application screenshot — Stock Screen]

6. DFD for Collections & Payment Recording

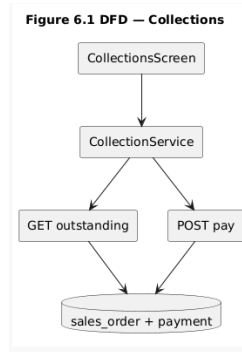
Figure 6.1 DFD Collections



[Insert rendered image for Figure 6.1]

6.1 Collections Screen

Figure 6.1.1 6.1 Collections Screen



[Insert rendered image for Figure 6.1.1]

The collections module supports Form 3 collection efficiency tracking. `CollectionService.fetchOutstandingInvoices()` calls `GET /api/collections/outstanding`, which aggregates `sales_order` totals minus `SUM(payment.amount)` per order, filtered by `visit.employee_id` for reps.

`fetchOutstandingBalance` uses `GET /api/collections/balance` for total outstanding amount displayed at the top of `CollectionsScreen`. `recordPayment` posts `orderId`, `amount`, `paymentMethod`, and `referenceNumber` to `/api/collections/pay`. The backend verifies that a rep can only pay orders linked to their own visits before inserting into the payment table.

The `Invoice` model class maps JSON fields `order_id`, `outlet_name`, `outstanding`, `due_date`, and `status` (Pending, Overdue, Paid) for list rendering.

Listing 6.1.1 — Flutter: CollectionService

```
import 'api_client.dart';
import '../models/invoice.dart';
import '../models/payment_record.dart';

class CollectionService {
  static Future<List<Invoice>> fetchOutstandingInvoices() async {
    try {
      final data = await ApiClient.get('/collections/outstanding');

      print('🟡 Invoices data type: ${data.runtimeType}');

      if (data is List) {
        return data
          .map((json) => Invoice.fromJson(json))
          .toList();
      } else {
        print('❌ Invoices data is not a List');
        return [];
      }
    } catch (e) {
      print('❌ fetchOutstandingInvoices error: $e');
      throw Exception('Failed to load Invoices: $e');
    }
  }

  static Future<double> fetchOutstandingBalance() async {
    try {
      final data = await ApiClient.getMap('/collections/balance');
      return _toDouble(data['total_outstanding']);
    } catch (e) {
      final invoices = await fetchOutstandingInvoices();

      return invoices.fold<double>(
        0,
        (sum, inv) => sum + inv.outstanding,
      );
    }
  }

  static Future<Map<String, dynamic>> recordPayment({
    required int orderId,
```

Listing 6.1.2 — Flutter: Invoice.fromJson

```

factory Invoice.fromJson(Map<String, dynamic> json) {
  return Invoice(
    orderId: _toInt(json['order_id']),
    orderNumber: _toString(json['order_number']),
    outletName: _toString(json['outlet_name']),
    total: _toDouble(json['total']),
    paidAmount: _toDouble(json['paid_amount']),
    outstanding: _toDouble(json['outstanding']),
    dueDate: _toString(json['due_date']),
    status: _toString(json['status']),
  );
}

```

Listing 6.1.3 — Backend: outstanding query (server.js)

```

const role = req.user.role;

if (!pool) {
  return res.json([
    {
      order_id: 1,
      order_number: '50-001',
      outlet_name: 'Toklo Maju Jaya',
      total: 330000,
      paid_amount: 0,
      outstanding: 330000,
      due_date: '2026-05-30',
      status: 'Pending',
    },
  ]);
}

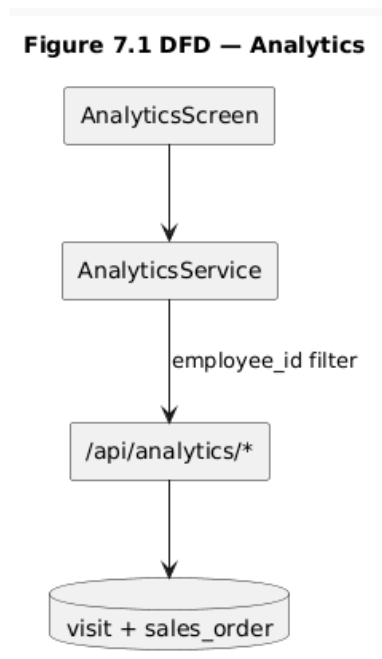
try {
  let query, params;

  if (role === 'rep') {
    query = `
      SELECT
        so.order_id,
        so.order_number,
        o.outlet_name,
        so.total,
        COALESCE(SUM(p.amount), 0) AS paid_amount,
        so.total - COALESCE(SUM(p.amount), 0) AS outstanding,
        (so.order_date + INTERVAL '30 days')::date AS due_date,
        CASE
          WHEN so.total - COALESCE(SUM(p.amount), 0) <= 0 THEN 'Paid'
          WHEN (so.order_date + INTERVAL '30 days') < NOW() THEN 'Overdue'
          ELSE 'Pending'
        END AS status
      FROM sales_order so
      JOIN visit v
        ON v.visit_id = so.visit_id
      JOIN outlet o
        ON o.outlet_id = v.outlet_id
      LEFT JOIN payment p
        ON p.order_id = so.order_id
    `;
  }
}

```

7. DFD for Analytics & Merchandising

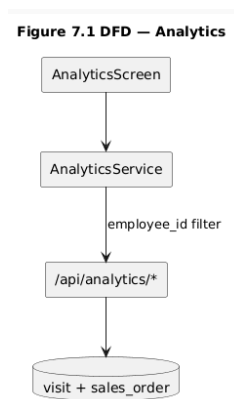
Figure 7.1 DFD Analytics and Merchandising



[Insert rendered image for Figure 7.1]

7.1 Analytics Screen

Figure 7.1.1 7.1 Analytics Screen



[Insert rendered image for Figure 7.1.1]

AnalyticsScreen presents sales performance using the `fl_chart` package. AnalyticsService calls four endpoints: `/api/analytics/daily-sales`, `top-products`, `top-outlets`, and `summary`. All queries join through `visit` and filter by `employee_id` so each rep sees only their own performance unless a manager passes `rep_id` (admin analytics on `RepDetailScreen`).

`resolveAnalyticsEmployeeId()` in `server.js` centralizes this logic: reps always use their JWT `employee_id`; managers may pass query `rep_id`.

Listing 7.1.1 — Flutter: AnalyticsService.fetchSummary

```
static Future<Map<String, dynamic>> fetchSummary() async {  
  try {  
    return await ApiClient.getMap('/analytics/summary');  
  } catch (e) {  
    print('✗ fetchSummary error: $e');  
    return {};  
  }  
}
```

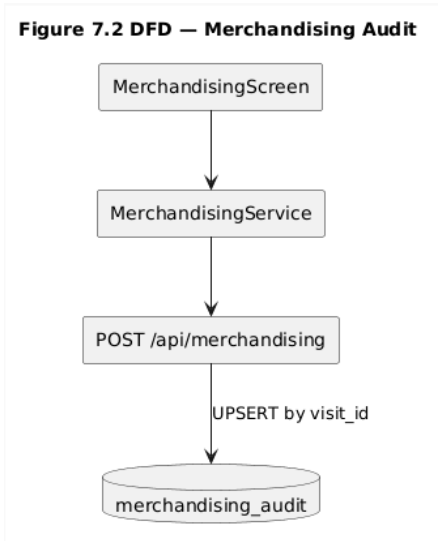
Listing 7.1.2 — Backend: analytics summary SQL

```
try {
  const result = await pool.query(
    `
    SELECT
      COALESCE(SUM(so.total), 0) AS total_sales,
      COUNT(DISTINCT so.order_id) AS total_orders,
      COUNT(DISTINCT v.visit_id) AS total_visits,
      CASE
        WHEN COUNT(DISTINCT v.visit_id) > 0 THEN
          (
            COUNT(
              DISTINCT CASE
                WHEN so.order_id IS NOT NULL THEN v.visit_id
              END
            )::float
            / COUNT(DISTINCT v.visit_id)
          ) * 100
        ELSE 0
      END AS strike_rate
    FROM visit v
    LEFT JOIN sales_order so
      ON so.visit_id = v.visit_id
    WHERE
      v.employee_id = $1
      AND v.visit_date >= CURRENT_DATE - INTERVAL '30 days'
    `,
    [employeeid],
  );

  res.json(
    result.rows[0] || {
      total_sales: 0,
      total_orders: 0,
      total_visits: 0,
      strike_rate: 0,
    },
  );
}
```

7.2 Merchandising Audit

Figure 7.2 7.2 Merchandising Audit



[Insert rendered image for Figure 7.2]

MerchandisingScreen captures in-store execution quality per Form 3 merchandising requirements. MerchandisingService.submit() sends visitId, planogram_score (0–100), shelf_share, posm_placement, and notes to POST /api/merchandising. The merchandising_audit table has UNIQUE(visit_id) so one audit row exists per visit; PostgreSQL ON CONFLICT DO UPDATE implements upsert semantics.

Before insert, the server checks visit.employee_id matches the JWT for rep role to prevent cross-user data tampering.

Listing 7.2.1 — Flutter: MerchandisingService

```
import 'api_client.dart';

class MerchandisingService {
  static Future<Map<String, dynamic>?> fetchByVisit(int visitId) async {
    final data = await ApiClient.get('/merchandising/$visitId');

    if (data == null) return null;
    if (data is Map<String, dynamic>) return data;

    return null;
  }

  static Future<void> submit({
    required int visitId,
    required int planogramScore,
    required int shelfShare,
    required bool posmPlacement,
    String? notes,
  }) async {
    await ApiClient.post(
      '/merchandising',
      {
        'visitId': visitId,
        'planogram_score': planogramScore,
        'shelf_share': shelfShare,
        'posm_placement': posmPlacement,
        'notes': notes,
      },
    );
  }
}
```

Listing 7.2.2 — Backend: merchandising POST (server.js)

```

app.put(
  '/api/promotions/:id',
  authenticate,
  authorize('admin', 'manager'),
  async (req, res) => {
    const {
      description,
      type,
      conditions,
      reward,
      start_date,
      end_date,
      is_active,
    } = req.body;

    try {
      await pool.query(
        `
          UPDATE promotion
          SET
            description = $1,
            type = $2,
            conditions = $3,
            reward = $4,
            start_date = $5,
            end_date = $6,
            is_active = $7
          WHERE promotion_id = $8
        `,
        [
          description,
          type,
          JSON.stringify(conditions || {}),
          JSON.stringify(reward || {}),
          start_date,
          end_date,
          is_active !== undefined ? is_active : true,
          req.params.id,
        ],
      );
    }
  }
);

```

```
    } catch (err) {
      res.status(500).json({ error: err.message });
    }
  },
);

// =====
// MERCHANDISING AUDIT
// =====

app.post('/api/merchandising', authenticate, async (req, res) => {
  const {
    visitId,
    planogram_score,
    shelf_share,
    posm_placement,
    notes,
  } = req.body;

  const employeeId = req.user.employee_id;

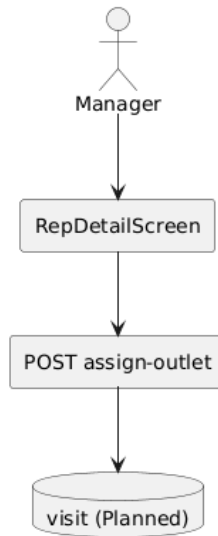
  if (!visitId) {
    return res.status(400).json({
      error: 'visitId required',
    });
  }

  if (!pool) {
    return res.json({
      success: true,
      message: 'Audit saved (mock)',
    });
  }
});
```

8. DFD for Admin, Outlet Management & Supervisor

Figure 8.1 DFD Admin Operations

Figure 8.1 DFD — Admin Assign Outlet



[Insert rendered image for Figure 8.1]

8.1 Admin Dashboard & Sales Rep CRUD

AdminDashboard lists all sales representatives via `AdminService.getSalesReps()` → GET `/api/admin/sales-reps`. Managers can add reps (POST `/api/admin/sales-reps`) which inserts `employee`, `user_account`, and `team_member` in one transaction. Soft delete sets `employee.is_active = false`. `RepDetailScreen` provides tabbed monitoring: dashboard clone, orders, payments, analytics, assigned outlets, and stock Excel upload.

This consolidates Form 3 web dashboard functionality into the same Flutter binary to reduce deployment complexity for Sprint 1–2.

Listing 8.1.1 — Flutter: `AdminService.getSalesReps`

```

import 'api_client.dart';
import '../models/sales_rep.dart';

class AdminService {
  static Future<List<SalesRep>> getSalesReps() async {
    try {
      final data = await ApiClient.get('/admin/sales-reps');

      if (data is List) {
        return data
          .map((json) => SalesRep.fromJson(json))
          .toList();
      }

      return [];
    } catch (e) {
      print('✗ getSalesReps error: $e');
      throw Exception('Failed to load sales reps: $e');
    }
  }
}

```

Listing 8.1.2 — Backend: list sales reps SQL

```

const result = await pool.query(`
  SELECT
    e.employee_id,
    e.name,
    e.email,
    e.phone,
    e.created_at AS joined_date,
    u.username,
    u.role,
    u.is_verified,
    e.is_active,
    COUNT(DISTINCT v.visit_id) AS total_visits,
    COALESCE(SUM(so.total), 0) AS total_sales
  FROM employee e
  INNER JOIN users u
    ON e.user_id = u.user_id
  LEFT JOIN visit v
    ON e.employee_id = v.employee_id
  LEFT JOIN sales_order so
    ON v.visit_id = so.visit_id
  WHERE u.role = 'rep'
  GROUP BY
    e.employee_id,
    e.name,
    e.email,
    e.phone,
    e.created_at,
    u.username,
    u.role,
    u.is_verified,
    e.is_active
  ORDER BY e.name ASC
`);

res.json(result.rows);

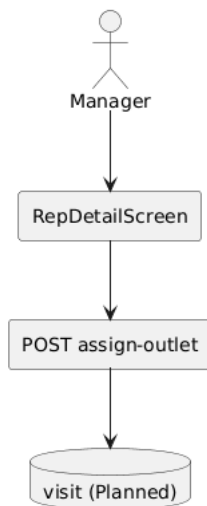
```

8.2 Assign Outlet to Sales Rep

Diagram source (PlantUML — render at <https://www.plantuml.com/plantuml> or VS Code

PlantUML extension, export PNG, insert image below):

Figure 8.1 DFD — Admin Assign Outlet



[Insert rendered image for Figure 8.2]

Route planning for a rep is implemented by creating rows in the visit table. POST `/api/admin/sales-reps/:id/assign-outlet` inserts visit (employee_id, outlet_id, visit_date, visit_time, status='Planned'). Default visit_date is CURRENT_DATE so the outlet appears on the rep dashboard immediately. Duplicate active assignments return HTTP 409. DELETE remove-outlet sets status to Missed for future planned visits.

GET `/api/admin/sales-reps/:id/outlets` returns distinct outlets with active visits; available-outlets returns outlets not yet assigned.

Listing 8.2.1 — Backend: assign outlet INSERT

```
const result = await pool.query(  
  `  
    INSERT INTO visit (  
      employee_id,  
      outlet_id,  
      visit_date,  
      visit_time,  
      status  
    )  
    VALUES (  
      $1,  
      $2,  
      $3,  
      $4,  
      'Planned'  
    )  
    RETURNING visit_id  
  `,  
  [repId, outlet_id, visitDate, visitTime],  
);  
  
console.log(  
  `✅ Outlet ${outlet_id} assigned to rep ${repId}, visit_id: ${result.rows[0].visit_id}`,  
);
```

Listing 8.2.2 — Backend: assign-outlet validation

```

return res.json({
  success: true,
  message: 'Outlet assigned successfully (mock)',
  visit_id: Date.now(),
});
}

try {
  const existingCheck = await pool.query(
    `
    SELECT
      v.visit_id,
      v.status
    FROM visit v
    WHERE
      v.employee_id = $1
      AND v.outlet_id = $2
      AND v.visit_date >= CURRENT_DATE
      AND v.status IN ('Planned', 'InProgress')
    `,
    [repId, outlet_id],
  );

  if (existingCheck.rows.length > 0) {
    console.log(
      `⚠️ Outlet ${outlet_id} already assigned to rep ${repId}`,
    );

    return res.status(409).json({
      error: 'Outlet already assigned to this sales rep',
    });
  }
}

```

8.3 Outlet CRUD & Settings / Profile

OutletScreen allows admin/manager to create outlets with GPS coordinates, credit_limit, priority (A/B/C), and store_type. Reps see a filtered list via GET /api/outlets?rep_id=. SettingsScreen loads profile through UserService.getProfile() → GET /api/auth/profile and supports PUT /api/auth/profile and POST /api/auth/change-password with bcrypt verification of the old password.

SupervisorDashboard calls GET /api/supervisor/team-dashboard, resolving manager_id from user_account for supervisor role so the supervisor sees the same team as their assigned manager.

Listing 8.3.1 — Backend: PostgreSQL pool (db.js)

```
const { Pool } = require('pg');
require('dotenv').config();

// Konfigurasi koneksi dari file .env
const pool = new Pool({
  host: process.env.DB_HOST,
  port: process.env.DB_PORT,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
});

// Cek koneksi saat pertama kali
pool.connect((err, client, release) => {
  if (err) {
    console.error(
      '❌ Gagal konek ke PostgreSQL:',
      err.message,
    );
    console.error(
      'Periksa file .env (DB_HOST, DB_USER, DB_PASSWORD, DB_NAME)',
    );
    console.error(
      'Database harus sudah dibuat dan script SQL diimport di pgAdmin.',
    );
  } else {
    console.log(
      '✅ PostgreSQL: ${process.env.DB_NAME} @ ${process.env.DB_HOST}:${process.env.DB_PORT}',
    );
    release();
  }
});

module.exports = pool;
```

Listing 8.3.2 — Flutter: UserService

```
import 'api_client.dart';

class UserService {
  static Future<Map<String, dynamic>> getProfile() async {
    return ApiClient.getMap('/auth/profile');
  }

  static Future<Map<String, dynamic>> updateProfile({
    required String name,
    required String email,
    required String phone,
  }) async {
    return ApiClient.putMap(
      '/auth/profile',
      {
        'name': name,
        'email': email,
        'phone': phone,
      },
    );
  }

  static Future<Map<String, dynamic>> changePassword({
    required String oldPassword,
    required String newPassword,
  }) async {
    return ApiClient.postMap(
      '/auth/change-password',
      {
        'oldPassword': oldPassword,
        'newPassword': newPassword,
      },
    );
  }
}
```

Listing 8.3.3 — Backend: change password

```

app.post('/api/auth/change-password', authenticate, async (req, res) => {
  const { oldPassword, newPassword } = req.body;
  const employeeId = req.user.employee_id;

  if (!oldPassword || !newPassword) {
    return res.status(400).json({
      error: 'Old and new password required',
    });
  }

  if (newPassword.length < 4) {
    return res.status(400).json({
      error: 'Password minimal 4 karakter',
    });
  }

  if (!pool) {
    return res.json({
      success: true,
    });
  }

  try {
    const result = await pool.query(
      `
      SELECT password_hash
      FROM user_account
      WHERE employee_id = $1
      `,
      [employeeId],
    );

    if (result.rows.length === 0) {
      return res.status(404).json({
        error: 'User not found',
      });
    }

    const valid = await bcrypt.compare(
      oldPassword,
      result.rows[0].password_hash,
    );

    if (!valid) {
      return res.status(401).json({
        error: 'Password lama salah',
      });
    }
  }

```

Database Implementation

Database implementation is based on the design explained in Form 3, Part B, executed as PostgreSQL schema `sales_canvassing`.

Database implementation follows Form 3 relational design, deployed as PostgreSQL database `sales_canvassing`. The canonical schema script is `sales_canvassing.sql` executed once in pgAdmin 4. Connection parameters are stored in `sales_canvassing_backend/.env` and loaded by `db.js` through the pg Pool module.

Master data tables (`section`, `employee`, `user_account`, `team`, `team_member`, `unit`, `product`, `outlet`) support organizational structure and catalog management. Transactional tables (`visit`, `sales_order`, `order_detail`, `payment`, `merchandising_audit`, `promotion`) capture field activity. The stock table enforces per-rep inventory with `UNIQUE(product_id, type, employee_id)`. Views `v_sales_dashboard` and `v_team_performance` accelerate reporting queries.

Sample seed data includes users `admin`, `manager`, `repa`, `repub`. Password hashes must be regenerated with `npm run setup-passwords` because SQL placeholders are not valid bcrypt strings.

Figure DB.1 Entity Relationship Diagram

Diagram source (PlantUML — render at <https://www.plantuml.com/plantuml> or VS Code PlantUML extension, export PNG, insert image below):

Listing DB.1 — `user_account` table

```

CREATE TABLE user_account (
    user_id SERIAL PRIMARY KEY,
    employee_id INTEGER UNIQUE NOT NULL
        REFERENCES employee(employee_id),

    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,

    role VARCHAR(20) NOT NULL
        CHECK (role IN ('admin', 'manager', 'supervisor', 'rep')),

    is_verified BOOLEAN DEFAULT FALSE,
    last_login TIMESTAMP
);

```

Listing DB.2 — visit and sales_order

```

CREATE TABLE visit (
    visit_id SERIAL PRIMARY KEY,

    employee_id INTEGER NOT NULL
        REFERENCES employee(employee_id),

    outlet_id INTEGER NOT NULL
        REFERENCES outlet(outlet_id),

    visit_date DATE NOT NULL DEFAULT CURRENT_DATE,

    check_in_time TIMESTAMP,
    check_in_lat DECIMAL(10, 8),
    check_in_lng DECIMAL(11, 8),

    status VARCHAR(20) DEFAULT 'Planned'
);

CREATE TABLE sales_order (
    order_id SERIAL PRIMARY KEY,

    order_number VARCHAR(20) NOT NULL UNIQUE,

    visit_id INTEGER NOT NULL
        REFERENCES visit(visit_id),

    order_type VARCHAR(20) NOT NULL,

    subtotal DECIMAL(10, 2),
    discount DECIMAL(10, 2),
    tax DECIMAL(10, 2),
    total DECIMAL(10, 2)
);

```

Listing DB.3 — stock_history trigger function (excerpt)

CREATE OR REPLACE FUNCTION log_stock_change()

RETURNS TRIGGER AS \$\$

```

BEGIN

IF (OLD.quantity != NEW.quantity) THEN

INSERT INTO stock_history (product_id, employee_id, type,
old_quantity, new_quantity, change_amount, changed_by)
VALUES (NEW.product_id, NEW.employee_id, NEW.type,
OLD.quantity, NEW.quantity, NEW.quantity - OLD.quantity, NEW.updated_by);

END IF;

RETURN NEW;

END;

$$ LANGUAGE plpgsql;

```

User Interface Implementation

User interface implementation uses Flutter Material Design widgets. A single APK/binary serves all roles; navigation trees differ by role as described in Section 1.2. Key screens and their service dependencies:

Screen	Primary Service / Package	Purpose
LoginScreen	auth_service.dart	Entry point; form validation
RoleBasedNavigator / MainNavigation	main.dart	Drawer navigation for rep
DashboardScreen	dashboard_service.dart	Visit plan + KPI cards
VisitsScreen	visit_service.dart	Today's visits, missed, reschedule
RoutePlanningScreen	api_client.dart, flutter_map	Map visualization

GpsTrackingScreen	geolocator, api visits checkin/out	Geo-fenced attendance
OrderTakingScreen	order_service.dart, promotion_service.dart	Cart and checkout
StockScreen	stock_service.dart	Per-rep inventory
CollectionsScreen	collection_service.dart	Outstanding and payments
AnalyticsScreen	analytics_service.dart, fl_chart	Charts and rankings
MerchandisingScreen	merchandising_service.dart	In-store audit form
OutletScreen	outlet_service.dart	CRUD for admin; filtered list for rep
SettingsScreen	user_service.dart	Profile and password
AdminDashboard/ RepDetailScreen	admin_service.dart	Team and outlet assignment
SupervisorDashboard	api_client.dart	Team KPIs
PromotionsScreen/ ProductsAdminScreen	promotion_service.dart	Master data

Hardware Implementation

No custom IoT circuit board is required. Hardware consists of standard Android smartphones with GPS, gyroscope, and network connectivity for REST API calls. Development hardware is a

PC/laptop hosting PostgreSQL and Node.js. Form 3 planned offline SQLite on device — hardware storage sufficient; software implementation deferred to Sprint 2.

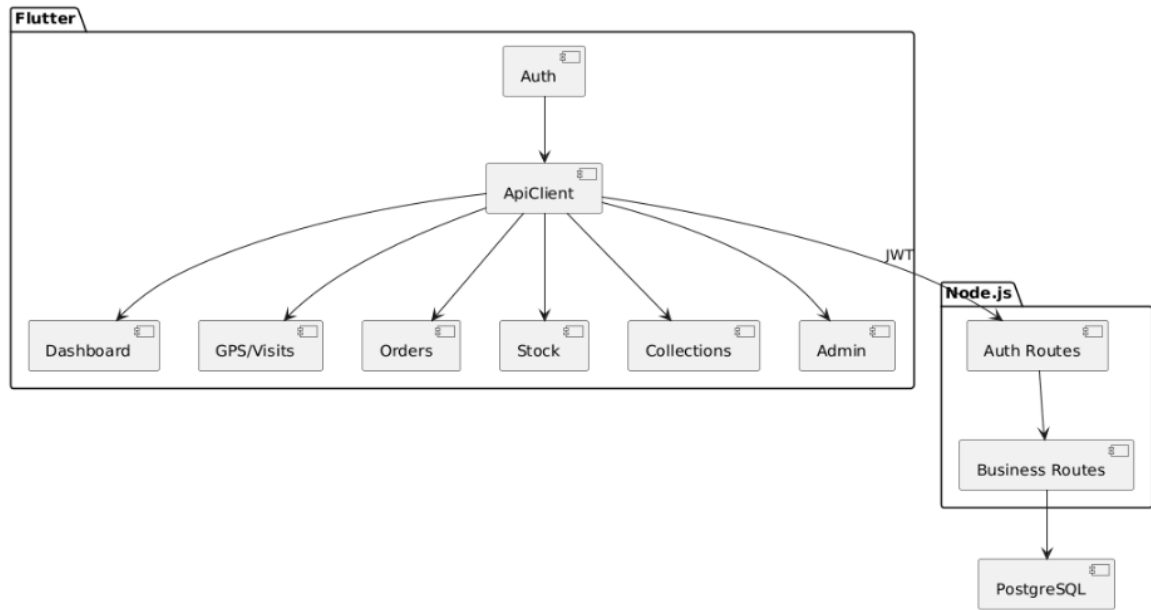
Integration Among Every Module

Integration is achieved through a layered architecture. The presentation layer (Flutter widgets) never accesses PostgreSQL directly; all persistence flows through REST endpoints in server.js. ApiClient centralizes HTTP verbs, JSON encoding, and Authorization headers. The authenticate middleware decodes JWT on every protected route and attaches req.user. Business modules share the same pool connection from db.js. Cross-module business rules include: (1) visit must exist before order for Sales/Return; (2) payment must reference order owned through visit.employee_id; (3) stock rows keyed by employee_id; (4) merchandising_audit references visit_id. Planned Form 3 integrations (Redis cache, RabbitMQ message queue, ERP REST API, offline sync queue) will attach at the API layer without changing Flutter screen contracts.

Figure INT.1 Module Integration Diagram

Diagram source (PlantUML — render at <https://www.plantuml.com/plantuml> or VS Code PlantUML extension, export PNG, insert image below):

Figure INT.1 Integration Between Modules



[Insert rendered image for Figure INT.1]

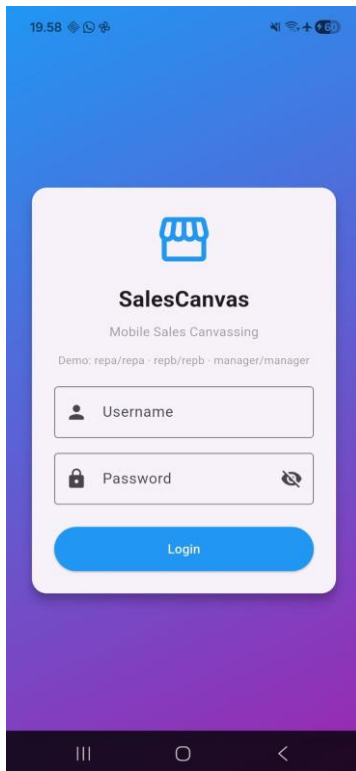
PRODUCT DISPLAY

SOFTWARE PRODUCT DISPLAY

The following figures show the running Mobile Sales Canvassing Application. Capture screenshots from flutter run on emulator or physical device after backend npm start and successful login. Include both success paths and error dialogs where noted.

Figure P.1.1 Login Screen

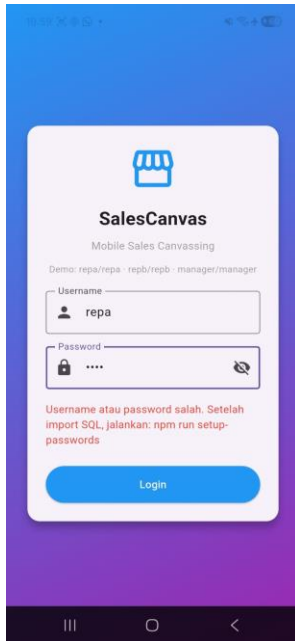
Initial authentication UI with SalesCanvas branding.



[Insert application screenshot — Login Screen]

Figure P.1.2 Login Failed

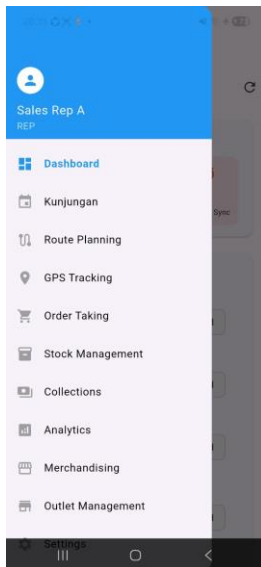
Validation or credential error displayed to user.



[Insert application screenshot — Login Failed]

Figure P.1.3 Login Success — Rep Main Navigation

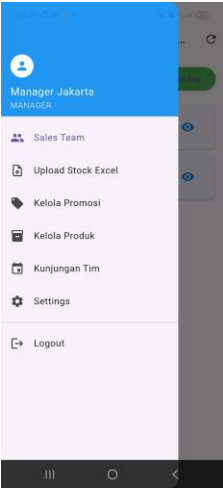
Drawer menu with all rep modules.



[Insert application screenshot — Login Success — Rep Main Navigation]

Figure P.1.4 Login Success — Admin Dashboard

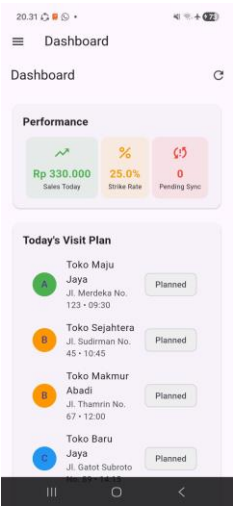
Manager view after manager/manager login.



[Insert application screenshot — Login Success — Admin Dashboard]

Figure P.2.1 Dashboard — Visit Plan

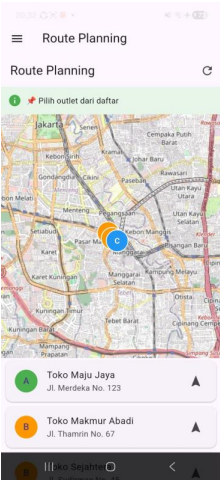
List of today's planned outlets with priority.



[Insert application screenshot — Dashboard — Visit Plan]

Figure P.3.1 Route Planning Map

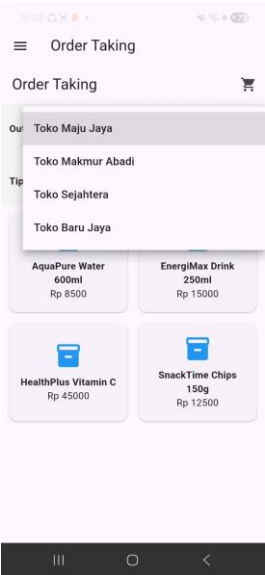
OpenStreetMap with outlet markers.



[Insert application screenshot — Route Planning Map]

Figure P.4.1 Order Taking — Product List

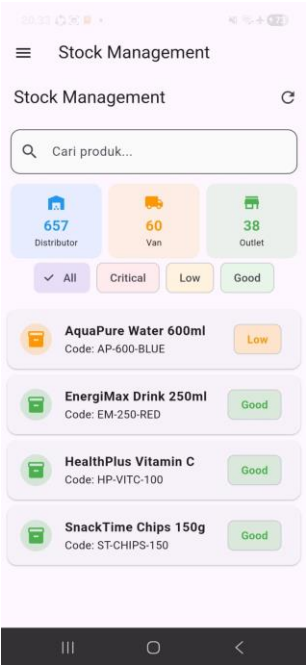
Products with price and add-to-cart.



[Insert application screenshot — Order Taking — Product List]

Figure P.5.1 Stock — Status

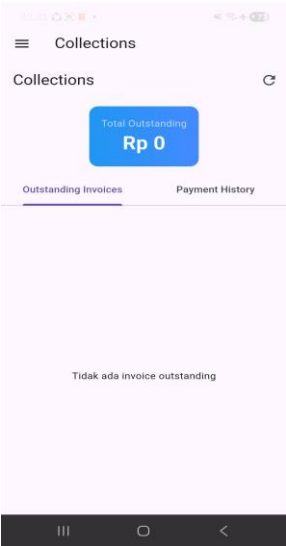
Green indicators for adequate van stock.



[Insert application screenshot — Stock — Good Status]

Figure P.6.1 Collections — Outstanding

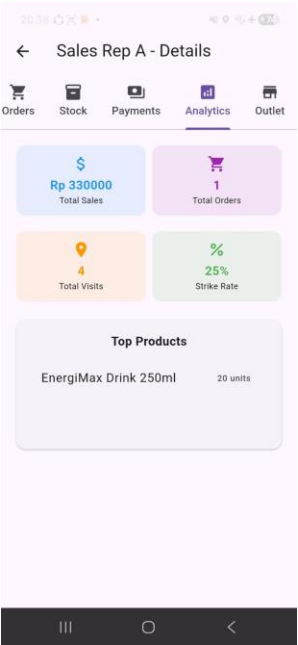
Unpaid orders with amounts.



[Insert application screenshot — Collections — Outstanding]

Figure P.7.1 Analytics — Daily Sales Chart

fl_chart line graph.



[Insert application screenshot — Analytics — Daily Sales Chart]

Figure P.8.1 Merchandising Audit Form

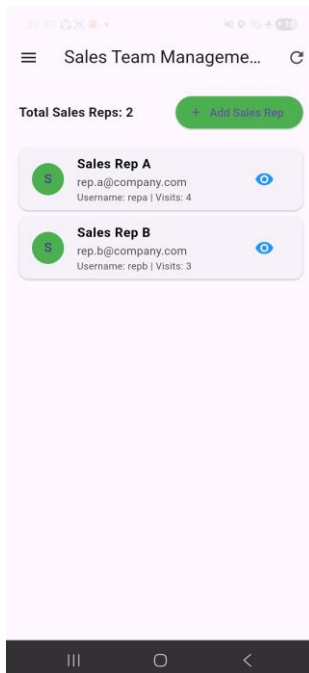
Planogram score and POSM toggle.



[Insert application screenshot — Merchandising Audit Form]

Figure P.9.1 Admin — Sales Rep List

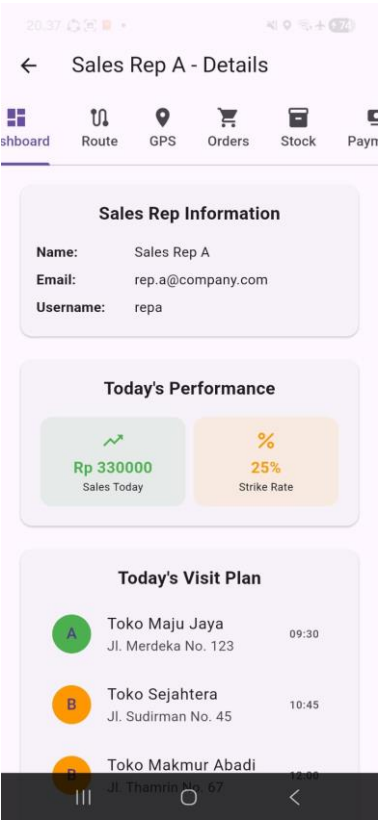
All reps with totals.



[Insert application screenshot — Admin — Sales Rep List]

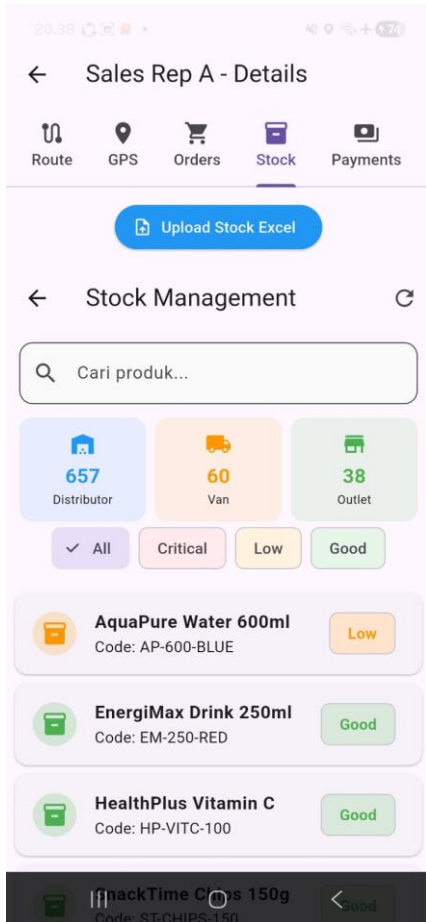
Figure P.9.2 Rep Detail — Assign Outlet

Available vs assigned outlets.



[Insert application screenshot — Rep Detail — Assign Outlet]

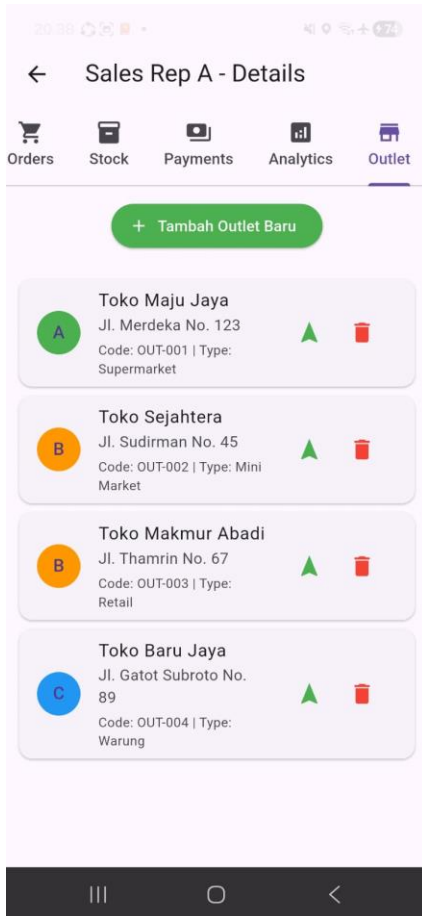
Figure P.9.3 Rep Detail — Stock Excel Upload



File picker and upload result.

[Insert application screenshot — Rep Detail — Stock Excel Upload]

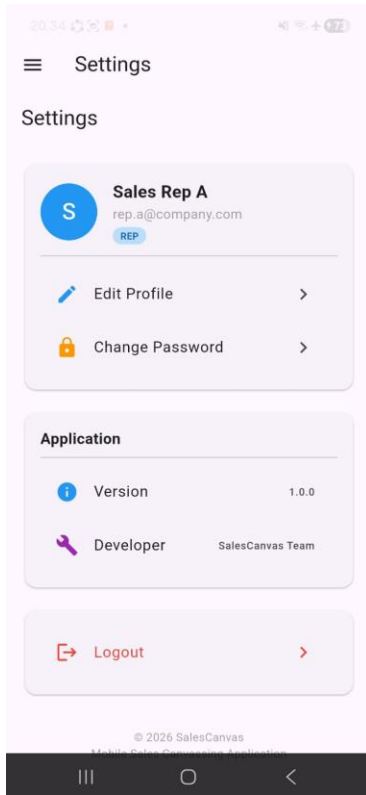
Figure P.10.1 Outlet Create Form



Admin creating new outlet with GPS.

[Insert application screenshot — Outlet Create Form]

Figure P.10.2 Settings Profile



User profile edit dialog.

[Insert application screenshot — Settings Profile]

COMPONENT COST ANALYSIS

All components required to develop and test the project. Production server hosting (VPS/cloud) may be added separately for deployment.

Category	Item	Qty	Unit Price	Subtotal
Software	Flutter SDK	1	Rp 0	Rp 0
Software	Node.js, Express, PostgreSQL, pgAdmin 4	1	Rp 0	Rp 0
Software	Android Studio / VS Code	1	Rp 0	Rp 0
Service	OpenStreetMap (flutter_map)	1	Rp 0	Rp 0
TOTAL				Rp 0

FUNCTIONAL TESTING

Functional testing scenarios derived from Form 3 Design Part D (Implementation and Testing Scenario), mapped to the implemented codebase. Each feature has a separate table. Fill Output Result after execution.

Table D.1 — User Login

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Valid credentials	repa / repa	HTTP 200, JWT, navigate to Rep UI	
2	Invalid password	repa / wrong	HTTP 401, error on LoginScreen	
3	Empty username	blank / repa	Client validation message	
4	Deactivated employee	inactive user	HTTP 401 Account deactivated	
5	Backend stopped	valid creds, no server	Network error with API URL hint	

Table D.2 — Role-Based Access

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Rep accesses	GET /admin/sales-	HTTP 403	

	admin API	reps as rep	Forbidden	
2	Manager lists reps	GET /admin/sales- reps as manager	JSON array of reps	
3	Rep dashboard scope	GET /dashboard as repa	Only repa visits	

Table D.3 — GPS Check-In

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Inside geofence	lat/lng near outlet	success, visit InProgress	
2	Outside geofence	lat/lng 200m away	HTTP 400 Outside geofence	
3	No GPS permission	permission denied	Error on GpsTrackingScreen	

Table D.4 — Order Taking

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Sales after	items, Sales type	order_number,	

	check-in		DB rows	
2	No check-in	Sales without check_in	HTTP 400	
3	FOC order	orderType FOC	total = 0	
4	Credit exceeded	large cart	HTTP 400 credit limit	
5	Return order	orderType Return	outstanding decreased	

Table D.5 — Stock

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Rep own stock	GET /stock	employee_id from JWT	
2	Admin view rep 3	GET /stock?rep_id=3	Rep B quantities	
3	Excel upload	valid xlsx	successCount > 0	

Table D.6 — Collections

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	List outstanding	rep login	only own orders	
2	Record payment	amount, Cash	payment row inserted	
3	Wrong rep order	other rep orderId	HTTP 403	

Table D.7 — Admin Assign Outlet

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Assign today	outlet_id, rep 3	visit_id, Planned	
2	Duplicate	same outlet again	HTTP 409	
3	Visible on dashboard	login as rep	outlet on plan	

Table D.8 — Merchandising

No.	Scenario	Every Possible Input	Expected Output	Output Result

1	Submit audit	visitId, scores	audit_id returned	
2	Wrong visit owner	other rep visitId	HTTP 403 for rep	

Table D.9 — Offline Mode (Planned Form 3)

No.	Scenario	Every Possible Input	Expected Output	Output Result
1	Local SQLite sync	N/A current build	Planned Sprint 2	

MANUAL GUIDE

SYSTEM BUILD DOCUMENTATION FROM THE SOURCE

Project: Mobile Sales Canvassing Application

Repository structure:

- sales_canvassing_app/ — Flutter mobile application
- sales_canvassing_backend/ — Node.js Express REST API
- sales_canvassing.sql — PostgreSQL schema and seed data

Step 1 — Install Prerequisites

Install Flutter: <https://docs.flutter.dev/get-started/install>

Install Node.js LTS: <https://nodejs.org/>

Install PostgreSQL and pgAdmin 4

Run flutter doctor and resolve Android toolchain issues

Step 2 — Database Setup (pgAdmin 4)

Open pgAdmin → Create database sales_canvassing

Query Tool → Open file sales_canvassing.sql → Execute (F5)

If trigger fails on older PostgreSQL: run scripts/fix-stock-trigger.sql

```
cd sales_canvassing_backend
```

```
npm install
```

Copy .env.example to .env and set DB_PASSWORD

```
npm run setup-passwords
```

Step 3 — Run Backend API

`npm start`

Browser: `http://localhost:3000/api/test` → database: connected

Optional POST `http://localhost:3000/api/auth/login` with `{"username":"repa","password":"repa"}`

Step 4 — Run Flutter Application

`cd ../sales_canvassing_app`

`flutter pub get`

Android emulator: `flutter run` (API host defaults to 10.0.2.2)

Physical phone on same Wi-Fi: `flutter run --dart-define=API_HOST=<your_PC_IP>`

Login repa / repa for sales rep testing

Login manager / manager for admin dashboard testing

Step 5 — Build Release APK

`flutter build apk --release --dart-define=API_HOST=<production_server_ip>`

END-USER SYSTEM INSTALLATION

Hardware Requirements

- Android 10 or newer smartphone
- GPS and internet (Wi-Fi or mobile data)

Software Requirements

- Install APK provided by IT/admin
- Allow location permission when prompted

USER GUIDE PER USER ROLE

Role	Login	Main Functions	Step-by-Step Usage
Sales Representative (rep)	repa / repa	Dashboard, visits, GPS, orders, stock, collections, analytics, merchandising	1) Login 2) Open Dashboard — review visit plan 3) Route Planning — see map 4) GPS Tracking — Check-in at outlet 5) Order Taking — add products, submit 6) Check-out 7) Collections — record payment 8) Merchandising — fill audit
Manager / Admin	manager / manager or admin / admin	Manage reps, assign outlets, upload stock Excel, products, promotions	1) Login 2) Admin Dashboard — select rep 3) Rep Detail — Assign outlet or Upload stock 4) Create outlet in Outlet Management 5) Review orders/payments tabs

Supervisor	supervisor / supervisor (if configured in DB)	Monitor team performance	1) Login 2) Supervisor Dashboard — review team visit and sales totals
------------	---	-----------------------------	---

REFERENCES

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.

Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures.